

入門 Cloud Run

Google Cloud / Cloud Run を、AWS との違いから学ぶ

TypeScript + Hono / Artifact Registry / Cloud Run

目次

はじめに

0. 事前準備

座学パート

1. AWSとGoogle Cloudの考え方の違い

2. Cloud Runとは

ハンズオンパート

3. Google Cloudのセットアップ

4. Dockerのおさらいとコンテナ起動

5. Cloud Runへデプロイする

6. 更新とロールバック

7. カナリアリリースとタグ付きURL

8. オートスケールを観察する

9. ログとメトリクスをのぞく

10. ソースデプロイ

おわりに

99. 後片付け

入門 Cloud Run

AWS を実務で使っているエンジニアに向けた、Google Cloud / Cloud Run のハンズオン教材です。

この教材のゴール

1. **Cloud Run を通して Google Cloud の良さを知る**
    - 「デプロイしたら HTTPS の URL が返ってくる」体験から、Google Cloud の SaaS 的なアプローチを体感する
2. **AWS との違いを知る**
    - 単なるサービス名の読み替えではなく、思想の違いを理解する
3. **異なるクラウドの目線を持ち帰る**
    - 普段 AWS で組んでいる構成を「Google Cloud ならどう組むか」で考えられるようになる

対象者

- **AWS を実務で利用したことがある方**
    - Google Cloud は未経験、もしくは BigQuery などを少し触った程度
- **コンテナ技術は知識として知っている方**
    - Docker・devcontainer・ECS・Kubernetes
- **クラウドベンダーの設計思想の違いを学びたい方**
    - AWS と Google Cloud の違いをサービス名の対応ではなく、設計思想の違いとして理解したい方

進め方

- 前半(1〜2章)は座学パートになります。
    - ハンズオン当日スライドもあるため、そちらも参照ください。
    - 実践!Cloud Run ハンズオン!! / Cloud Run handson - Speaker Deck
- 後半(3章〜)はハンズオンです。
    - ブラウザ(Cloud Shell エディタと Google Cloud コンソール)だけで完結します。
    - 手元のPCへのアプリやCLIのインストールは不要です。

今回持ち帰って欲しい3つのポイント

今回持ち帰って欲しいポイントと、ハンズオンが終わったら最後にまた返ってきて見直していただきたいポイントでもあります。

1. **AWSはビルディングブロック、Google CloudはSaaS的アプローチ**
    - Cloud Run は deploy 1コマンドの裏で、LB・証明書・スケーリング・ログ収集を「しなかった」
2. **イメージの不変性がプラットフォームまで貫かれている**
    - リビジョン・ロールバック・カナリアリリース・タグ付きURL

3. **従量課金と常駐のダイヤルを回せる**
    - スケールtoゼロ・オートスケール（min/max-instances）・concurrency（同時接続数/スロットリング）

AWS や他のクラウドベンダーでもこの目線は使えます。
「この構成、Cloud Run 的な発想ならどう簡単にできるか?」「ECS・Lambda でなら?」のような別のクラウドの視点を知ることは、いま使っているクラウドをより良く使うことにつながります。

かかる費用

Cloud Run・Artifact Registry ともに無料枠が大きく、このハンズオンの範囲であればほぼ無料枠に収まります(数円〜数十円程度)。
心配な場合は最後の後片付けを必ず実施するか、ハンズオン専用プロジェクトを作って終了後にプロジェクトごと削除してください。

Repository

- https://github.com/y-ohgi/introduction-CloudRun

0. 事前準備

手元のPCへのインストールは不要です。
ハンズオンはすべてブラウザ(Google Cloud コンソールとCloud Shell エディタ)で完結します。

用意するもの

Google Cloud アカウント

会社のGoogle Workspaceアカウントではなく、個人アカウントを推奨します。
会社アカウントは、管理者設定でAPIが無効化されていたり、組織ポリシーによって進行できない箇所が出てくる可能性があります。

Google Chrome などのモダンブラウザ

シークレット(プライベート)モードは避けてください。
サードパーティCookieが無効だと Cloud Shell エディタが開けないことがあります。

トラブルシューティング

- `gcloud services enable` **で課金エラーが出る** — プロジェクトに課金アカウントが紐付いていません。課金ページから紐付けてください。
- **Cloud Shell エディタが開かない・真っ白になる** — シークレットモードやサードパーティCookieのブロックが原因のことが多いです。通常モードのブラウザで開き直すか、エディタを新しいウィンドウで開いてください。
- `--allow-unauthenticated` **が権限エラーで失敗する** — 会社アカウントの組織ポリシー(ドメイン制限共有)が原因の可能性が高いです。個人アカウント+個人プロジェクトに切り替えてください。
- **IAM 権限を付与した直後に 403 が返る** — IAM の変更の反映は通常2分程度、場合によっては5分以上かかります。数分待ってからリトライしてください。
- **Cloud Shell が切断された・タブを閉じてしまった・環境変数が消えた** — ファイルは `$HOME` パス配下に置くので消えずに残っており、失われるのは `export` を実行する環境変数だけです。**どの章にいても**、5章の「0. 環境変数の準備」を再実行すれば復元できます。

途中で分からなくなったら

1. **その手順の直後にある「成功していれば」を読む。**
    - 5章以降の主要な手順の後には「 **成功していれば** 」(そこまでで何が表示されていれば正解か)が付いています。まずここと見比べるのが最短です。
    - 5章の「環境変数の準備」には「 **詰まったら** 」(よくある原因と復旧コマンド)も付いています。
2. **環境変数を疑う。**
    - コマンドが `--region=` や `-docker.pkg.dev//` のように途中が空で失敗する場合は、Cloud Shell の再接続で環境変数が消えています。上の「Cloud Shell が切断された」の項目を見てください。
3. **同じコマンドを、そのまま再実行する。**

- この教材のコマンドは、何度実行しても同じ結果になるように作ってあります。
- 例えば、`gcloud run deploy` は新しいリビジョンを作るだけ、`ALREADY_EXISTS` は「すでにある」という意味なのでそのまま次へ進んで構いません。

4. **章の頭からやり直す。**
   - それでも状態が分からなくなったら、その章の最初の手順から順にコピペし直すのが確実です。
   - 各章は前章の完了状態を前提にしているので、前の章の手順を先に済ませてください。

5. **リポジトリをcloneしなおす。**
   - 3章の「4. サンプルコードを手元に用意する」のコードブロックを再実行すると、`~/cloudrun-handson` が初期状態に戻ります。
   - 6章以降で書き換えた `src/index.ts` も元に戻るので、必要ならその章の書き換えからやり直してください。

1. AWSとGoogle Cloudの考え方の違い

1. ビルディングブロック vs SaaS的アプローチ

AWS の思想は**ビルディングブロック**です。
VPC・サブネット・セキュリティグループ・IAMロール・ALB・ターゲットグループ……小さな部品を積み上げて、自分のアーキテクチャを組み立てます。自由度が高い一方で、「Webアプリを1つ公開する」だけでも組み立てる部品の数は多くなります。そしてこの部品の数は、作るときだけの話ではありません。動かしたあとに設定を読み、壊れたときに開く対象の数にもなります。

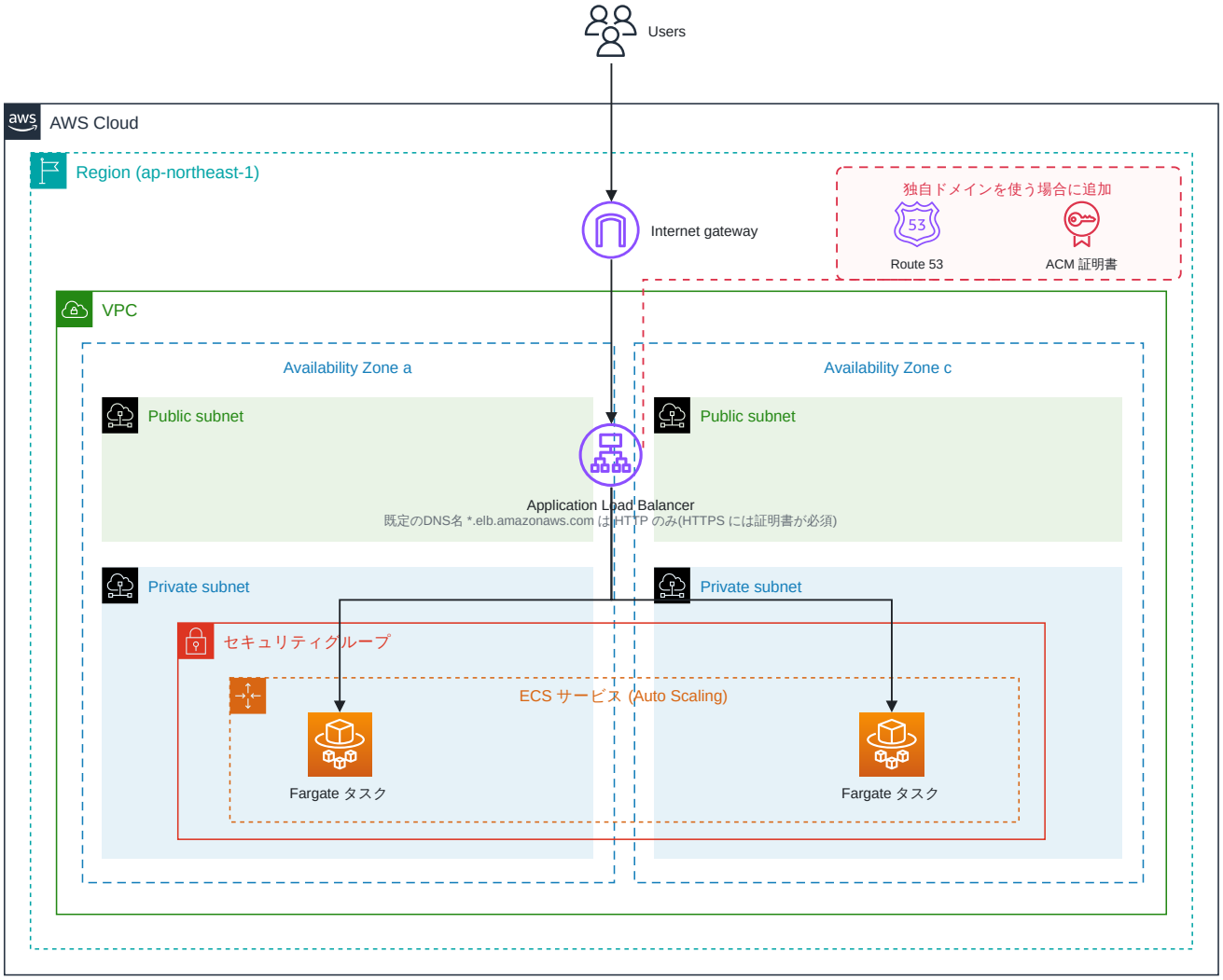
Google Cloud のマネージドサービスは、どちらかというと **SaaS 的なアプローチ**です。
「Webアプリを公開したい」という目的に対して、統合済みのサービスが1つ用意されていて、デフォルトで実用的な設定になっています。部品を組む代わりに、完成品の設定を必要な分だけ調整します。調整できる範囲は狭くなりますが、ラーニングコストが低く始めやすいです。

この違いが一番わかりやすいのが、今回扱う Cloud Run です。
まずは「コンテナを1つ、HTTPSで公開するまで」に必要なものを並べ、そのあとに運用の目線でもう一度見比べます。

2. コンテナを1つ、HTTPSで公開するまで

AWS (ECS on Fargate + ALB)

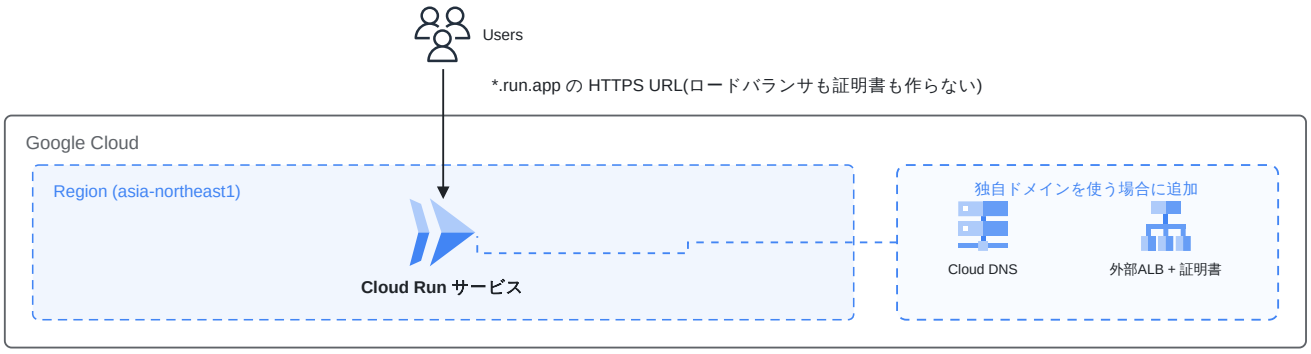
既定のDNS名で公開: 10 要素(HTTP のみ)



※ リスナー・ターゲットグループ・タスク定義は ALB / ECS の設定項目のため、図では枠とアイコンを持たない

Google Cloud (Cloud Run)

既定のURLで公開: 1 要素(HTTPS 込み)



独自ドメインの条件は両者とも同じ(DNS + 証明書が必要)。分かれ目は既定のURLで HTTPS が使えるかどうか

	AWS (ECS on Fargate + ALB)	Google Cloud (Cloud Run)
ネットワーク	VPC·サブネット·SG を用意	不要(必要なら後から VPC 接続)
負荷分散	ALB + ターゲットグループ + リスナー	不要(組み込み)

	AWS (ECS on Fargate + ALB)	Google Cloud (Cloud Run)
TLS証明書	HTTPSリスナーには証明書が必須。ALB の既定DNS名 (`*.elb.amazonaws.com`)は HTTP のみ	不要(`*.run.app` のHTTPS URLが自動発行)
オートスケール	Application Auto Scaling を設定	デフォルトで有効(0台〜既定100台)
ログ	awslogs ドライバや FireLens を設定	不要(stdout が自動で Cloud Logging へ)
デプロイ	タスク定義 + サービス更新	`gcloud run deploy` コマンド or yaml で詳細定義
独自ドメイン	Route 53 等のDNS + ACM 証明書	Cloud Runのドメインマッピング機能 or Cloud DNS + GLB

> 表の「不要」は「自分で作らなくてよい」という意味で、「設定できない」ではありません。

既定で払い出される URL で HTTPS が使えるか*です。*Cloud Run* の `.run.app` は HTTPS 込みですが、ALB の既定DNS名は HTTP のみで、HTTPS リスナーを作るには証明書が必須です(AWS: Create an HTTPS listener)。
証明書はドメイン名と一致していなければならないため、実質「所有ドメインが前提」になります。

最初からHTTPSで使えることによるメリットは、bot・マイクロサービス間通信・web-hookに繋がります。

どちらが優れているという話ではありません。
細かく制御したいなら AWS 的なアプローチが強く、素早く価値を出したいなら Google Cloud 的なアプローチが強い。
両方の目線を持っていると、状況に応じて最適な選択ができる——それが今日のゴールです。

> **発展コラム**
> AWS 側にも1コマンドで立つ経路があります。急ぐ場合はこの節を読み飛ばして、次の「運用で向き合うプロダクトの数」へ進んで構いません。

ここまで部品の数を並べましたが、「最初の1コマンドで公開できるか」はもう両クラウドの差ではありません。
2025年11月に GA した **ECS Express Mode** は、1回の API 呼び出しで ECS クラスタ・タスク定義・サービス・ALB(HTTPSリスナー・リスナールール・ターゲットグループ)・セキュリティグループ・サービスリンクロール・Application Auto Scaling・ロググループ・メトリクスアラーム・ACM証明書 を作り、`https://<service>.ecs.<region>.on.aws` の HTTPS URL を返します(AWS: Resources created by Amazon ECS Express Mode services)。
用意するのはコンテナイメージと IAM ロール2つだけで、Express Mode 自体に追加料金はありません。

ただし、作られた10種類は**自分のアカウントのリソース**です。

- 壊れたときに読むのは、これまでどおり ALB のアクセスログ・ターゲットグループのヘルス状態・タスクの停止理由
- ALB の設定とデプロイ戦略は Express Mode の API からは変更できず、独自ドメインは API の外で ALB を直接編集する
- VPC 内で最初に作った Express Mode サービスが、その VPC の ALB のスキームとアベイラビリティゾーンを決める。ALB は最大25サービスで共有される
- 既定値のうち確認しておきたいもの: ALB のアクセスログは無効、Desync 緩和モードは無効、パブリックサブネットではタスクにパブリックIPが付く

- オートスケールの既定は CPU使用率60%目標で、最小1タスク・最大20タスク

AWS 自身がこう書いています。「Instead of managing **hundreds of configuration parameters across multiple services**」。
つまり設定項目は消えていません。既定値が入っただけです。しかも既定値の一覧は「Express Mode から変えられるもの」と「**変えたいなら元のサービスへ行くもの**」に分かれています。

作る手数は減っても、理解する対象は減りません。
だからこの教材では、比べる軸を「作るときの手数」ではなく「運用で向き合うプロダクトの数」に置きます。次の節がその比較です。

運用で向き合うプロダクトの数

Cloud Run を「ECS + ALB が1つになったもの」と捉えると、いちばん体感が変わるのは障害調査です。
同じことを知りたいときに、どこを開くかを並べてみます。

知りたいこと	AWS (ECS on Fargate + ALB)	Google Cloud (Cloud Run)
500 が返る原因の切り分け	ALB のアクセスログ → ターゲットグループのヘルス状態 → ECS のタスク一覧と停止理由 → CloudWatch Logs のログストリーム	「ログ」タブ(リクエストのログとアプリの stdout が同じ画面に並ぶ・9章)
負荷が増えたときの挙動	ターゲットグループのメトリクス + Application Auto Scaling のポリシー + ECS のサービスイベント	「指標」タブのインスタンス数と、同時実行数の設定(8章)
直前の変更を戻す	タスク定義のリビジョンを選び直して ECS サービスを更新	リビジョン一覧でトラフィックの割り当てを戻す(6章・7章)
権限が足りない	タスク実行ロールとタスクロールを見分ける	Service に紐づくサービスアカウント1つ

AWS 側で開いた画面は ECS・ELB・CloudWatch・IAM・Application Auto Scaling に分かれています。
Cloud Run 側は Cloud Run の画面と、その先の Cloud Logging・Cloud Monitoring だけです。

覚えるつまみも数えられます。今日のハンズオンで触るのは次の範囲です。

- コンテナコントラクト(`PORT` を listen して stdout に出す・4章)
- 同時実行数 / CPU割り当て / min・maxインスタンス / リクエストタイムアウト(8章)
- リビジョンとトラフィック分割(6章・7章)

公平のために、Cloud Run 側の条件も揃える

「Cloud Run はリソース1つ」が成り立つのは `*.run.app` の URL で公開する場合です。これは Cloud Run の性質というより、Google が管理するドメインを借りていることの性質です。
独自ドメイン・WAF・複数リージョンへの振り分けといった本番の要件を足すと、Cloud Run の前に外部 ALB を置くことになり、Serverless NEG・バックエンドサービス・URLマップ・ターゲットHTTPSプロキシ・転送ルール・SSL証明書 の6つを自分で組みます(Google Cloud: Serverless NEG の概要)。
本番の要件を足せば、Google Cloud 側も部品を組む世界に戻ります。

この章で見せているのは「既定で払い出される URL まで」という同じ条件での比較です。

3. アカウント/プロジェクト構造の違い

AWS と大きく違うポイントです。
特に「プロジェクト」は Google Cloud を触るうえで最初に慣れておきたい概念です。
dev / staging / prod をプロジェクトで分ける、実験用に使い捨てプロジェクトを作る、といった運用が気軽にできます。

概念	AWS	Google Cloud
分離の単位	アカウント(Organizations の OU で階層化)	**プロジェクト**(組織 > フォルダ > プロジェクトの階層。フォルダは任意)
リージョンの扱い	コンソールでリージョンを切り替える	リソース作成時に指定(コンソールは全リージョン横断)
権限の主体	IAMユーザー / ロール	Google アカウント / **サービスアカウント**
API	常に有効	プロジェクトごとに**明示的に有効化**

AWS では「アカウント」が課金・リソースの分離単位でしたが、Google Cloud では1つのアカウント(Google Workspace であれば組織単位)の配下に **プロジェクト** を作り、それが分離単位になります。

AWSのアカウント = Google Cloudのプロジェクト

4. サービス対応表

ハンズオン中に「AWSでいうと何?」と思ったら、ここに戻ってきましょう。

今日使うサービス

Google Cloud	AWSでいうと	補足
Cloud Run	ECS Express Mode + ECS on Fargate + Lambda	どれとも微妙に違う。次章で詳しく。App Runner は**新規顧客への提供を終了**しており、AWS は ECS Express Mode への移行を案内している
Artifact Registry (GAR)	ECR	コンテナ以外(npm, Maven等)も置ける
Cloud Shell	CloudShell	エディタ付き・Docker が動く(AWS CloudShell も同様。ただし Docker は対応リージョン限定)
Cloud Build	CodeBuild	ソースデプロイの裏側で動く
Cloud Logging	CloudWatch Logs	設定不要で stdout を収集
Cloud Monitoring	CloudWatch	メトリクスはデフォルトで収集済み
Pub/Sub	SNS + SQS	1サービスで pub/sub もキューも担う

今日は使わないが対応を知っておくと便利なサービス

Google Cloud	AWSでいうと	補足
Compute Engine	EC2	
GKE	EKS	Kubernetes 発祥の地だけあり完成度が高い

Google Cloud	AWSでいうと	補足
Cloud Run functions(旧 Cloud Functions)	Lambda	現在は Cloud Run 基盤に統合され、名称も Cloud Run functions へ
Cloud Storage	S3	
Cloud SQL	RDS	
Spanner	Aurora DSQL(2025年5月GA)	グローバル分散RDB。強整合な分散SQLという点で近い
BigQuery	Redshift + Athena	Google Cloud 最大の看板サービス
Eventarc	EventBridge	各サービスのイベントを Cloud Run へ配送
Cloud Tasks	SQS(遅延キュー用途)	HTTPターゲットに直接push
Cloud Scheduler	EventBridge Scheduler	cron
Secret Manager	Secrets Manager	
Cloud Load Balancing	ALB/NLB + CloudFront の一部	グローバル単一エニーキャストIP
IAM サービスアカウント	IAM ロール	「アカウント」だが実体はロールに近い

まとめ

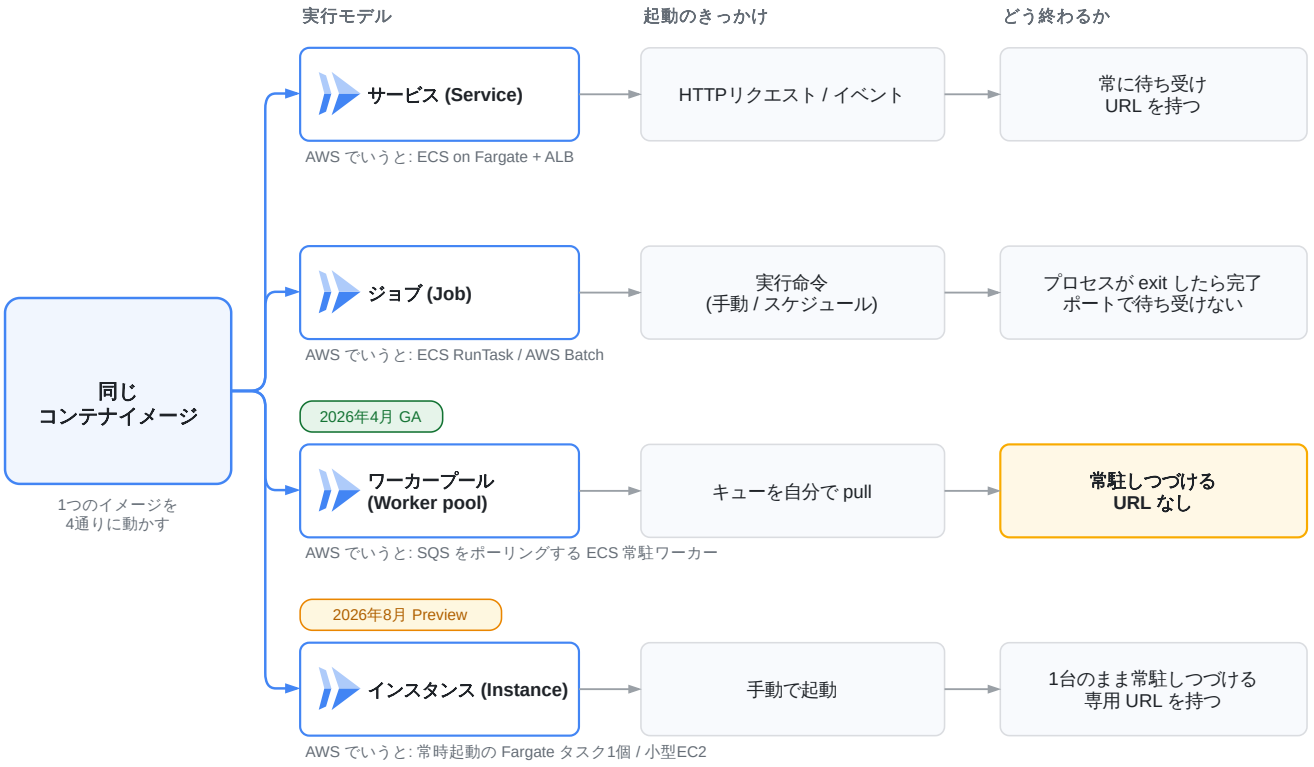
- 対応表は読み替えの入口。ただし思想が違うので1対1にはならない
- AWS は部品を組み立てる。Google Cloud は完成品を調整する
- **1コマンドで公開する経路はどちらのクラウドにもある。差が出るのは、運用で向き合うプロダクトの数**
- 分離単位は「アカウント」ではなく「プロジェクト」

2. Cloud Runとは

Cloud Run は「コンテナを渡すと、HTTPS の URL を返してくれる」フルマネージドなコンテナ実行基盤です。

- サーバー(VM・クラスタ)の管理は不要
- リクエストに応じて **0台から自動でスケール**し、使った分だけ課金
- デプロイはコンテナイメージを指定して行う

1.「4つの実行モデル」



デプロイ単位は4つとも同じコンテナイメージ。ただし待ち受けの作法は違い、サービスはポートで待ち受け、ジョブは待ち受けない。

Cloud Run には現在、ワークロードの形に合わせた**4つの実行モデル**があります。
今日のハンズオンの主役は Service ですが、この4分類を頭に入れておくと「Cloud Run は HTTP リクエストを受けるだけのもの」という思い込みをせずに済みます。

実行モデル	向いているもの	AWSでいうと
Service	HTTPリクエスト / イベント駆動	ECS on Fargate + ALB(1コマンドで立てるなら ECS Express Mode), Lambda + ALB
Job	実行して終わるバッチ処理	ECS RunTask, AWS Batch
Worker pool	常駐してキューをpullし続ける処理。Pub/Sub の pull 型ワーカーのような「リクエスト起点でない常駐処理」	SQSをポーリングするECS常駐ワーカー
Instance	1台であることに意味があるシングルトン常駐(2026年10月時点で Preview)	常時起動の Fargate タスク1個 / 小型EC2

同じイメージ・同じ開発体験のまま実行モデルだけ選び替えられるのが、Cloud Runの設計の一貫性です。

選び分けの軸は3つだけです。

1. リクエストが来る場合
    - Service
        - Webサービス向け
2. 処理に終わりがあるか
    - Job, Worker pool
        - タスク処理
3. コンテナインスタンスが1台か
    - Instance
        - botやAIエージェントなど状態を持つもの
        - 固有のURL・SSH接続・Cloud Storageにも対応

2. Cloud Run Service

このハンズオンでは、4つの実行モデルのうち **Cloud Run Service** を中心に扱います。
Cloud Run Serviceを学習するにあたり、まずは以下の3つの要素を押さえると良いでしょう。

1. **Service**
    - URL を持つ論理単位。ECS でいう「サービス+ALB+ターゲットグループ」を1つにしたもの
2. **リビジョン**
    - デプロイのたびに作られる**不変のスナップショット**(イメージ+環境変数+CPU/メモリ設定のセット)
    - 過去のリビジョンは残り続けるため、ロールバックなどが容易
3. **トラフィック分割**
    - どのリビジョンに何%流すかを制御
    - ロールバックもカナリアもこの仕組みの上に乗っている

> **「リビジョンが不変で、トラフィックを好きに振り分けられる」**
> Docker のイメージと同じ**「不変のスナップショットを積み重ねる」**という考え方が、プラットフォームの機能にまで貫かれています(後述の章で実際に手を動かしながら体感します)。
> ここが Cloud Run の設計の美しさでもあります。

3. 料金

課金モデルは2種類から選びます(Service 作成後も切替可)。

- **request-based billing(デフォルト)**: リクエスト処理中を中心に CPU・メモリが課金され、スケールtoゼロ時はインスタンス課金がなくなる。起動・終了処理や min-instances のアイドル時間など例外はある
- **instance-based billing**: インスタンス起動中は常時課金。単価はrequest-basedより安く、常時処理があるワークロード向け

無料枠も課金モデルごとにあります(2026年10月時点。単価・無料枠は改定されるので公式の料金ページが正)。

- request-based: 月あたり vCPU 18万秒 / メモリ 36万GiB秒 / リクエスト200万件
- instance-based: 月あたり vCPU 24万秒 / メモリ 45万GiB秒

個人開発や社内ツールなら実質無料で運用できることが多く、「とりあえず公開しておく」が気軽にできます。

4. AWSでいうと何?

Fargate・Lambda（・App Runnner）などのどれかに対応付けたくなりますが、どれとも微妙に違います。
この「微妙に違う」がそのまま Cloud Run の魅力なので、表で整理してみましょう(2026年10月時点)。

	Lambda(標準)	**ECS on Fargate + ALB**	**Cloud Run**
デプロイ単位	イメージ / ソース	イメージ	イメージ / ソース
実行モデル	1リクエスト=1実行環境 ※1	常駐	**1インスタンスで複数リクエストを並行処理**(既定80・最大1000)
スケールtoゼロ	○	×	○
課金	リクエスト処理時間	常時(起動時間)	**リクエスト単位 / 常時 を選択可**
実行時間上限	15分	なし	60分(リクエストあたり)
HTTPS URL	○ (Function URLs機能)	要 ALB + ACM	**○(自動発行)**
WebSocket / gRPC / HTTP2	△ (要別サービス)	○(要ALB/NLB設定)	○
VPC	**外(接続可)**	内で動く	**外(接続可)**

> ※1 2025年11月に登場した **Lambda Managed Instances** では、1つの実行環境で複数リクエストの並行処理が可能になりました。「普通のWebサーバー+インスタンス内並行処理」という Cloud Run が以前から持つモデルと近づいてきたと考えると両クラウドの設計思想の変化が見えて面白いところです。

押さえておきたいポイントは3つです。

1. **手軽さとコンテナの自由度。**
   - スケールtoゼロ・従量課金でありながら、中身はただのWebサーバー。
   - Lambda 特有のハンドラ関数やランタイム制約がなく、手元で `docker run` できるものがそのまま動きます。
2. **1インスタンスが複数リクエストを同時に捌く。**
   - 標準の Lambda は1リクエスト1環境なのでインスタンス数が爆発しがちですが、Cloud Run は「並行数(concurrency)」の分だけ1台で受けます。Webサーバーの常識がそのまま通用します。
3. **VPCが前提にない。**
   - AWS では最初に VPC を作りますが、Cloud Run はデフォルトで VPC の外。

    - VPC内のリソースへ接続が必要になったときだけ VPC に「つなぎに行く」発想です。

まとめ

- Cloud Run = コンテナを渡すと HTTPS URL が返ってくる。Lambda の運用感覚 × コンテナの自由度
- 実行モデルは「Service・Job・Worker pool・Instance」の4つ(Instance は Preview)。今日の主役は Service で、概念は「Service・リビジョン・トラフィック分割」の3つだけ。**AWS 側で ECS・ELB・CloudWatch・IAM・Application Auto Scaling に散っていた設定が、この3つと数個のつまみに集まっている**
- 座学はここまで。**ここからは全部手を動かします**

3. Google Cloudのセットアップ

Google Cloud のプロジェクトを作り、今日使う分だけAPIを有効化して、サンプルコードを手元に用意します。
ここから先はブラウザ(Google Cloud コンソールと Cloud Shell)だけで進みます。

1. Google Cloud プロジェクトを作る

1. Google Cloud コンソールにログイン
2. 初めての場合は無料トライアル(90日 / $300 クレジット)の案内に従う
3. 画面上部のプロジェクトセレクタから「新しいプロジェクト」を作成
    - プロジェクト名: `sandbox` など(IDは自動でユニークになります)
4. 課金が有効になっていることを確認(課金ページ)
    - クレジットカードの登録が必要です。

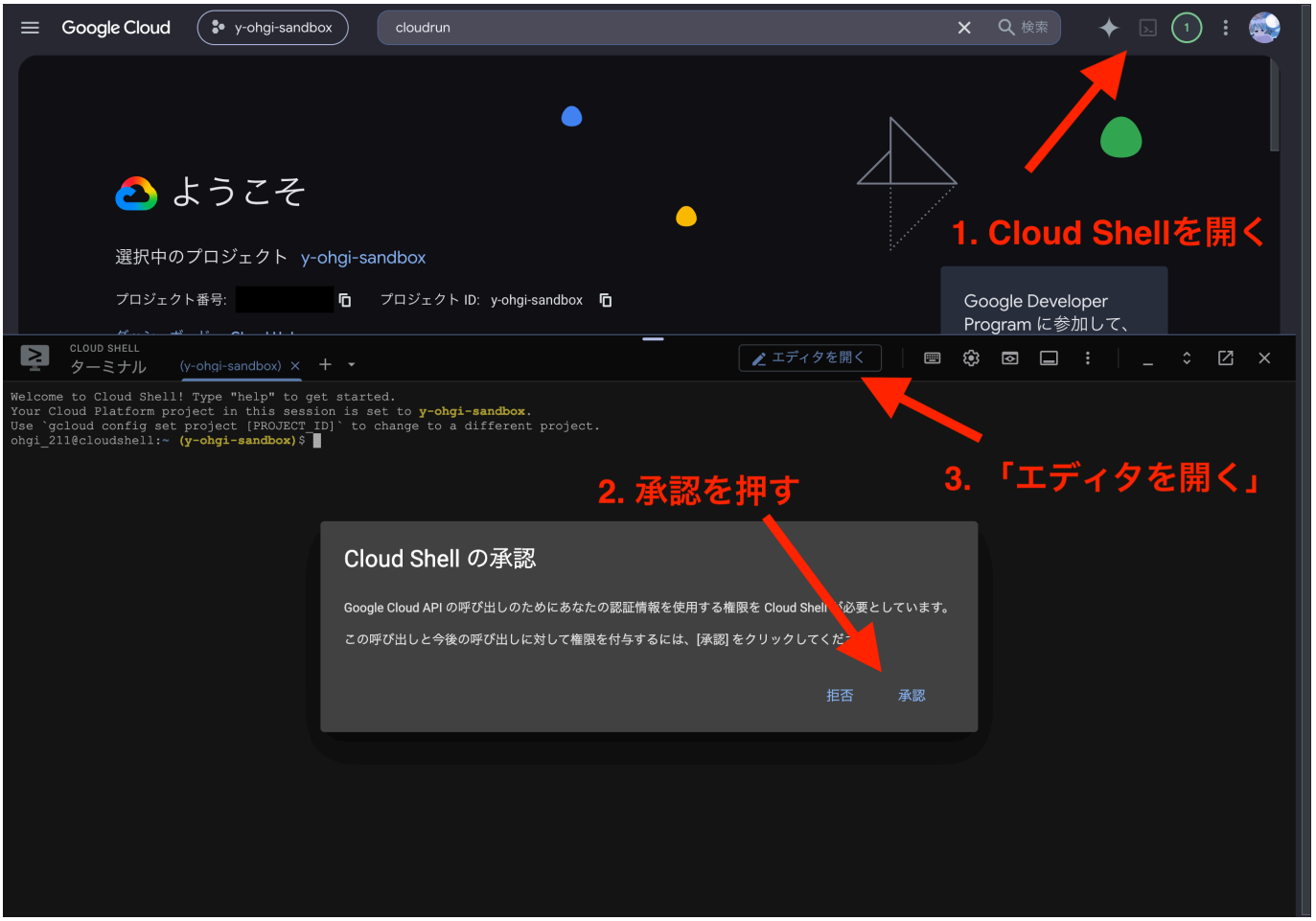
このハンズオンではハンズオン専用プロジェクトを推奨します。
ハンズオン後にプロジェクトごと削除すれば、消し忘れによる課金の心配がありません。

別のプロジェクトが選ばれたまま進むと、この先のコマンドがすべて無関係なプロジェクトに対して実行されてしまいます。
画面上部のプロジェクトセレクタで、今作ったプロジェクトが選ばれていることを確認してください。



2. Cloud Shellを開く

shell.cloud.google.com を開くか、コンソール右上のターミナルアイコンから Cloud Shell を起動し、「エディタを開く」をクリックします。

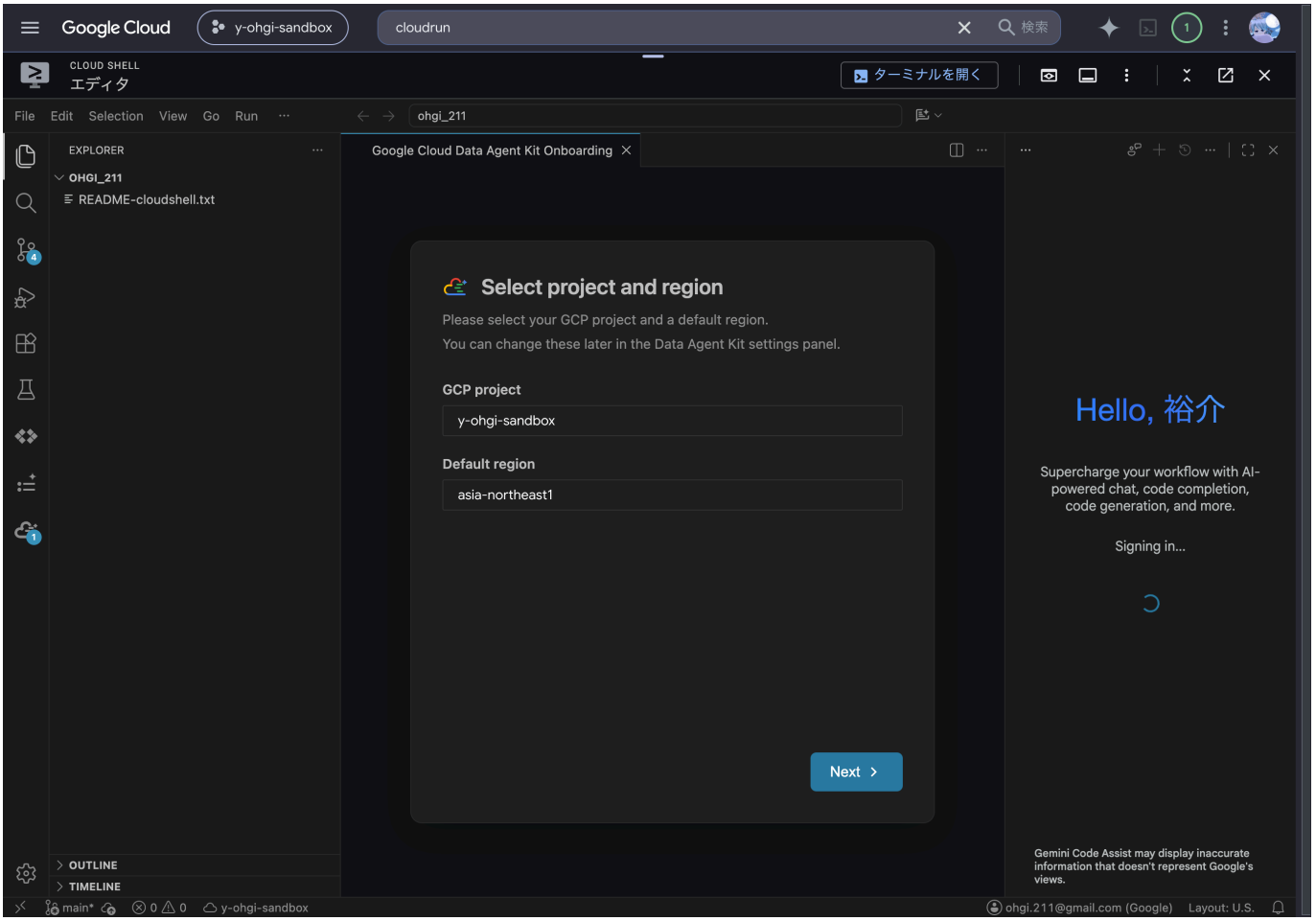


Cloud Shell は Google Cloud が無料で使える開発環境で、AWS の CloudShell に相当します。

3. Cloud Shell エディタを開く

Cloud Shellからエディタを開くと、VSCodeベースのエディタが使用可能です。
ファイルの編集・保存がそのままブラウザで完結します。
ハンズオンではこのエディタを使用して進めていきます。

初回起動時にプロジェクトとリージョンを聞かれます。
今回使用するプロジェクトと、デフォルトリージョンを `asia-northeast1` (東京リージョン)を選択します。

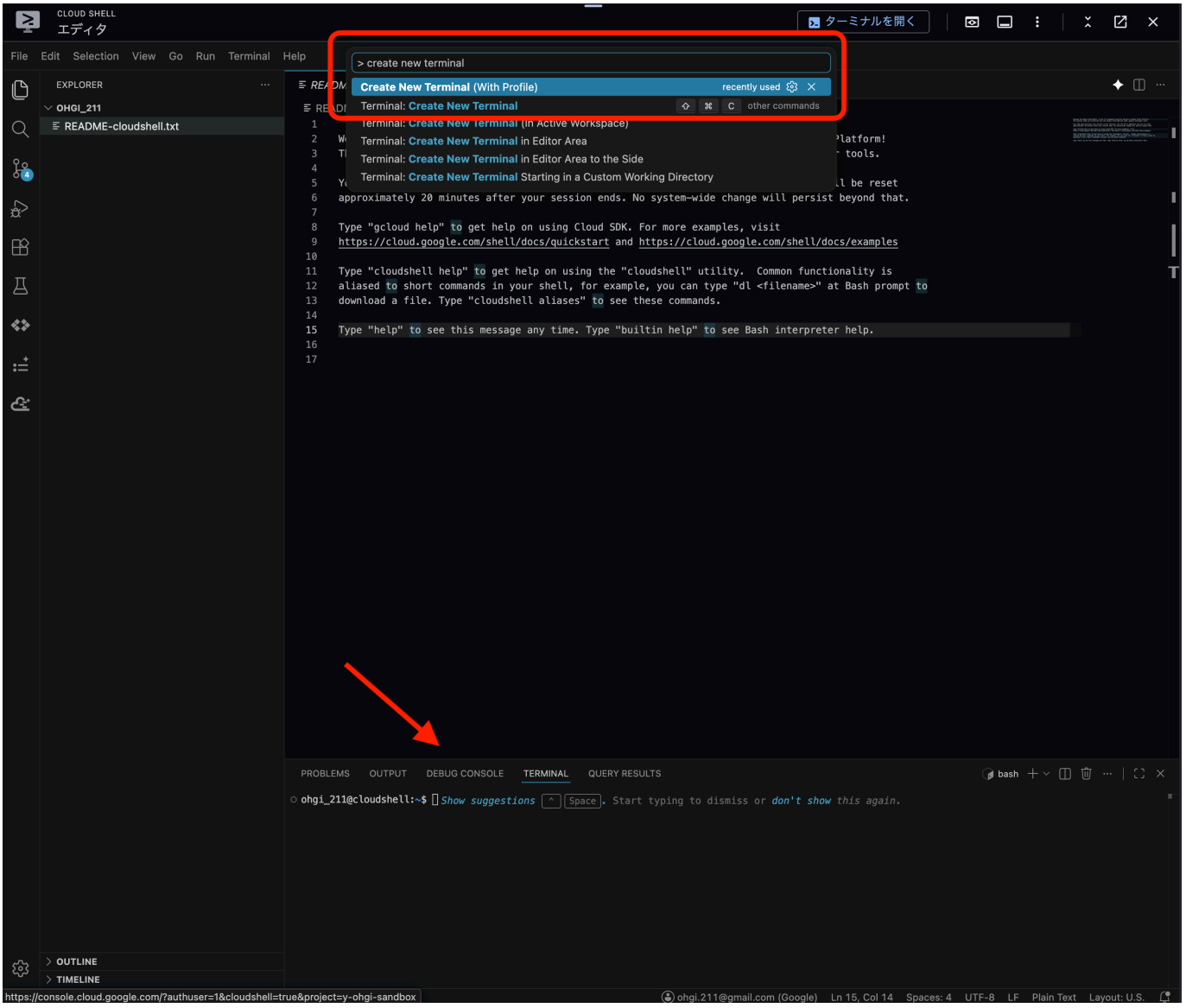


順調に進むとエディタが使用可能になります。

VSCodeを普段から使っている方なら直感的に使用可能だと思います。

今回欲しいのはエディタとTerminalなので、Terminalの起動も行います。

以下のように、コマンドパレットから `> create new terminal` と入力するとTerminalが画面下部に出力されます。



Docker

`docker` CLI が最初からインストールされています。
イメージのビルドと実行がその場でできます。今回のハンズオンの鍵です。

`gcloud` CLI

ログイン済みの状態で使えます。認証設定は不要です。

5GB の永続ホームディレクトリ

永続するのは `$HOME` だけです。それ以外への変更はセッション終了で失われます。

> **Cloud Shell を普段からよく使っている人へ**
> 週あたりの利用時間クォータ(デフォルト50時間)があり、使い切るとリセット時刻まで一切使えません。

4. 動作確認

```
gcloud config get-value project
docker version
```

プロジェクトIDが表示され、`docker version` がエラーにならなければ準備完了です。

`(unset)` と表示された場合や、作った覚えのないプロジェクトIDが表示された場合は、`gcloud` に今日使うプロジェクトが設定されていません。次のコマンドでプロジェクトを設定してから、もう一度 `gcloud config get-value project` を実行してください。

```
gcloud config set project <プロジェクトID>
```

5. 動作確認

```
gcloud config get-value project
docker version
```

プロジェクトIDが表示され、`docker version` がエラーにならなければ準備完了です。

`(unset)` と表示された場合や、作った覚えのないプロジェクトIDが表示された場合は、`gcloud` に今日使うプロジェクトが設定されていません。次のコマンドでプロジェクトを設定してから、もう一度 `gcloud config get-value project` を実行してください。

```
gcloud config set project <プロジェクトID>
```

プロジェクトIDはコンソール上部のプロジェクトセレクタで確認できます。
「1. Google Cloud プロジェクトを作る」で付けたプロジェクト名とは別に、`sandbox-123456` のように自動採番されたIDです。

覚えておくと速いショートカット

必須ではありませんが、知っておくと操作が速くなります。

操作	ショートカット
コマンドパレットを開く	`Cmd+Shift+P` / `Ctrl+Shift+P`
ファイルを名前で開く	`Cmd+P` / `Ctrl+P`
ファイル内を検索	`Cmd+F` / `Ctrl+F`
プロジェクト全体を検索	`Cmd+Shift+F` / `Ctrl+Shift+F`

このハンズオンの基本サイクルは、ここからずっとこの繰り返しになります。

1. エディタでファイルを編集する

2. 保存する(`Cmd+S` / `Ctrl+S`)
3. ターミナルでコマンドを実行する(`docker build` 、`gcloud run deploy` など)
4. 結果をブラウザやターミナルの出力で確認する

6. APIを有効化する

起動したTerminalから、今日使うAPIをまとめて有効化します。
数分かかることがあるので先に有効化を行います。

```
gcloud services enable \
  run.googleapis.com \
  artifactregistry.googleapis.com \
  cloudbuild.googleapis.com
```

1. run.googleapis.com
    - Cloud Runの有効化
2. artifactregistry.googleapis.com
    - ArtifactRegistryの有効化
        - コンテナレジストリです。
3. cloudbuild.googleapis.com
    - CloudBuild、(CI/)CDのためのサービスです。

AWSとの違い

Google Cloud では各サービスの API を明示的に有効化してから使います。
AWS のようにすべてのサービスが最初から呼べる状態ではなく、プロジェクトごとに使うサービスをオプトイン(/オプトアウト)する思想です。

まとめ

- Cloud Shell / Cloud Shell エディタの有効化をした。
- Google Cloud のプロジェクトを作成し、今日使うAPIを有効化した。
- ここから先はこのCloud Shell エディタを利用してハンズオンを進めていきます。

4. Dockerのおさらいとコンテナ起動

この章では Cloud Shell エディタとターミナルを行き来しながら、Docker の要点をおさらいし、実際に Web サーバーのコンテナイメージを作って・動かして・ブラウザで見るところまでやってみましょう。

> Docker をしっかり学びたい人は introduction-docker にまとまっています。今日は Cloud Run に必要な分だけ、駆け足で。

1. Dockerのおさらい: 3つの登場人物

用語	一言でいうと
Dockerfile	イメージの設計図(テキストファイル)
イメージ	アプリ+依存関係+OSライブラリを固めたスナップショット
コンテナ	イメージを実行したプロセス

ポイントは1つだけです: **イメージは不変(immutable)**。
「環境ごと固めて配る」からこそ、ローカルでも Cloud Run でも ECS でも同じように動きます。
この不変性が、ロールバックやカナリアリリースの土台になります。

2. サンプルコードを手元に用意する

今日使うサンプルコードをこのリポジトリから取得します。

```
cd ~
git clone https://github.com/y-ohgi/introduction-CloudRun.git
cd introduction-CloudRun/code/app
$ ls -a
# .  ..  Dockerfile  .dockerignore  package.json  src
```

アプリは TypeScript + Hono で書かれています。
Hono は Web 標準 API ベースの軽量な Web フレームワークです(Hono 自体は Cloudflare Workers や Deno などマルチランタイム対応ですが、今回は Node.js アダプタと Node.js API を使います)。
実行には Node.js 24 の組み込み **type stripping** (型注釈を剥がして実行する機能)を使い、この教材で使う範囲の TypeScript をビルドステップなしで動かします。
ビルドを1段消すことで、今日の主役である Cloud Run に集中するためです。

やり直すときは、cloneしたリポジトリを削除し、再度cloneすることで初期状態から再開できます。

3. Webサーバーを公開する

イメージをビルドする

```
docker build -t handson-app:v1 .
```

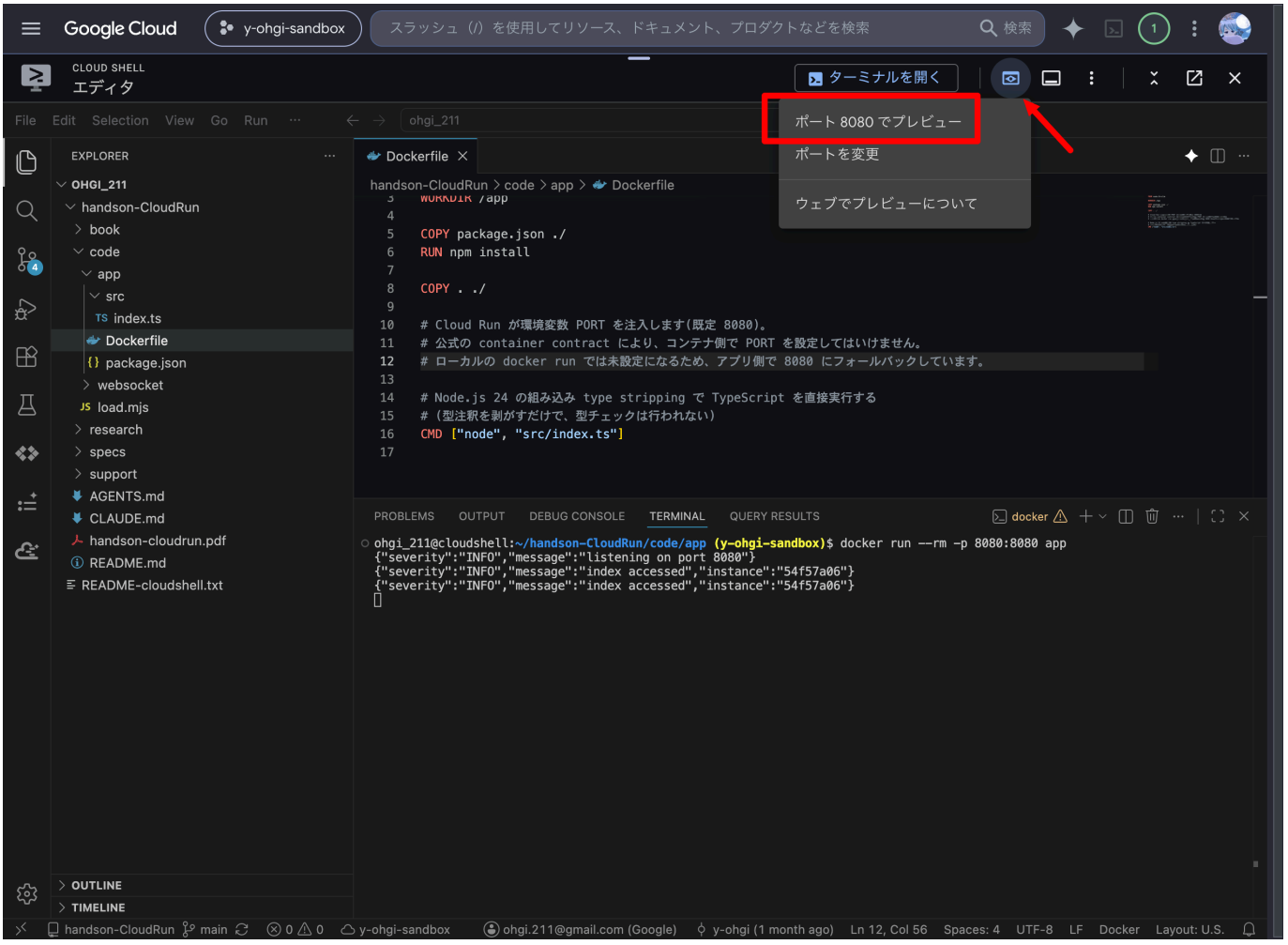
手元に Node.js がなくても(`npm install` を一度も打っていなくても)ビルドできることに注目してみましょう。
依存解決も実行環境も、すべてイメージの中で完結しています。
`docker images` で `handson-app` ができていることを確認してみましょう。

コンテナを起動する

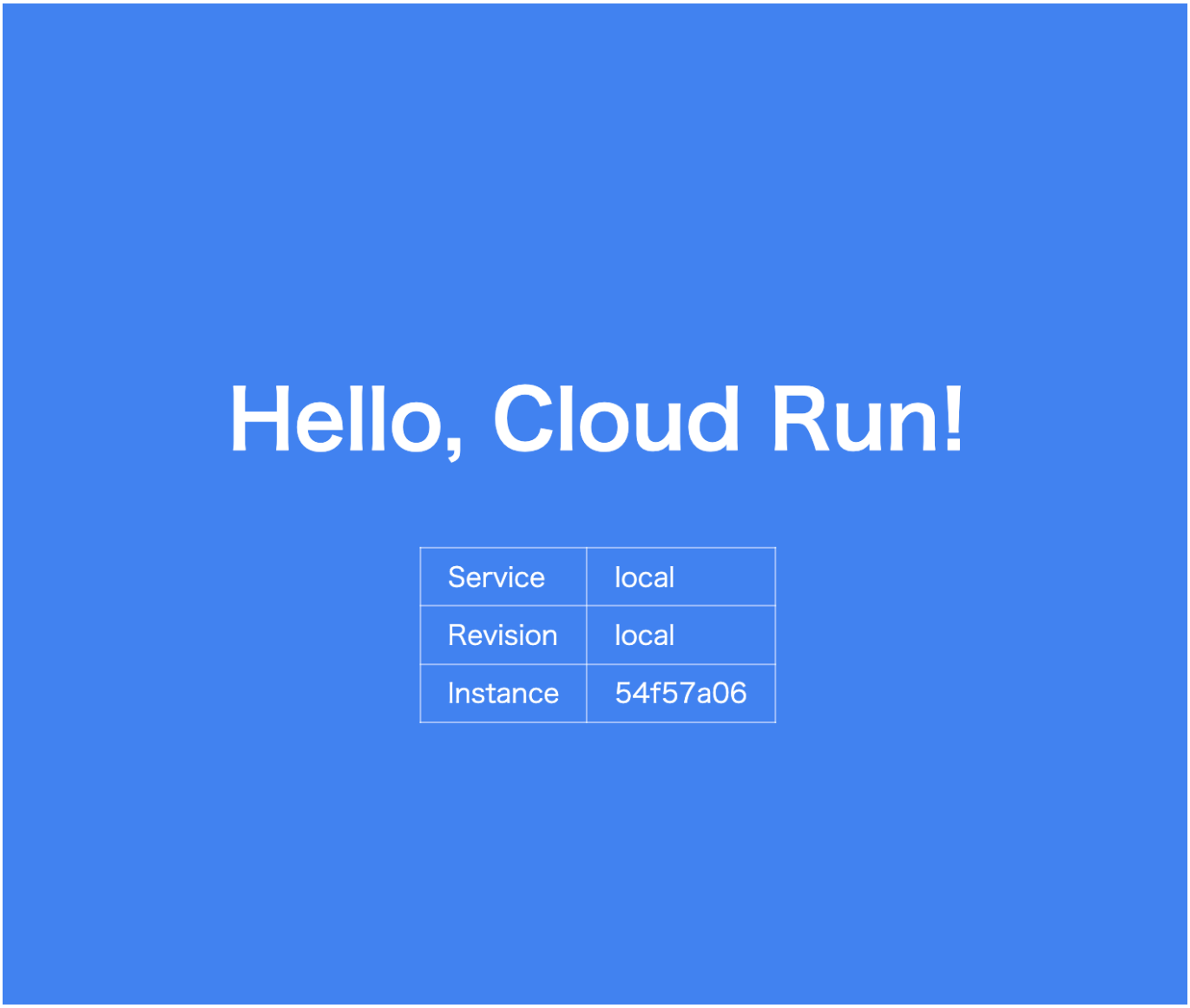
```
docker run --rm -p 8080:8080 handson-app:v1
```

ブラウザで見る

Cloud Shell の右上にある[ウェブでプレビュー]ボタン(ウィンドウの中に目が描かれたアイコン)→[ポート 8080 でプレビュー]をクリックします。



青い画面に「Hello, Cloud Run!」と表示されれば成功です。
Revision と Service が `local` になっていることも確認してみましょう——Cloud Run にデプロイすると、ここが自動で埋まります。



確認できたらターミナルで `Ctrl+C` を押してコンテナを止めます。

時間があれば

Dockerを普段使いしている方・ペースの早い方であれば時間が余ると思います。

Dockerfile を上から読んでみましょう。

ベースイメージ(`FROM`)に依存関係を重ね(`RUN npm install`)、コードを載せ(`COPY`)、起動コマンドを決める(`CMD`)。この積み重ねがそのままイメージのレイヤーになります。`package.json` だけ先に COPY しているのは、コードを変更しても `npm install` のレイヤーキャッシュが効くようにするテクニックです。

1点だけ Cloud Run 固有の約束があります: **環境変数 `PORT` で渡されたポートを listen すること**。ECS でいうタスク定義のポートマッピングにあたる取り決めが、この環境変数1つに集約されています。

コンテナの中に入ってみる

```
docker run --rm -it handson-app:v1 bash
ls        # 自分の書いたコードと node_modules が /app に入っている
```

```
node -v   # イメージに焼き込まれた Node.js 24
exit
```

まとめ

- エディタで編集 → 保存 → ターミナルで実行 → ブラウザで確認、の往復がこのハンズオンの基本サイクル
- Dockerfile → イメージ → コンテナ、の流れを一周した
- イメージは不変。だから「どこでも同じに動く」・「前のバージョンに戻せる」

5. Cloud Runへデプロイする

ここからはGoogle CloudとCloud Runを本格的に触っていきます。
4章で作ったイメージを Artifact Registry に push し、Cloud Run にデプロイして、世界中からアクセスできる HTTPS URL を手に入れます。

1. 環境変数の準備

この先の章でも使うので、まとめて設定しておきます(Cloud Shell が切断された場合はここから再実行してください)。

```
export PROJECT_ID=$(gcloud config get-value project)
export REGION=asia-northeast1   # 東京リージョン
export REPO=handson
export IMAGE=${REGION}-docker.pkg.dev/${PROJECT_ID}/${REPO}/app
```

成功していれば

`echo ${IMAGE}` で `asia-northeast1-docker.pkg.dev/<プロジェクトID>/handson/app` の形が表示されます。

詰まったら

`asia-northeast1-docker.pkg.dev//handson/app` のようにプロジェクトIDの部分が抜けている場合は、`gcloud` にプロジェクトが設定されていません。

以下を実行してから、この節のコードブロックを再実行してください。

```
gcloud config get-value project
gcloud config set project <プロジェクトID>
```

もしくはAPIの有効化を忘れている可能性があります。

```
gcloud services enable \
  run.googleapis.com \
  artifactregistry.googleapis.com \
  cloudbuild.googleapis.com
```

それでもダメ場合はログインをし直すことで解決できるパターンがあります。

```
gcloud auth login
```

2. Artifact Registry にリポジトリを作る

Artifact RegistryはDocker HubやECR と同じく、Google Cloudのコンテナレジストリのマネージドサービスです。
今回はArtifact Registryを使用してコンテナイメージを管理します。

以下のコマンドでコンテナイメージ用のリポジトリを作成します。

```
gcloud artifacts repositories create ${REPO} \
  --repository-format=docker \
  --location=${REGION} \
  --description="Cloud Run handson"
```

docker コマンドが Artifact Registry に push できるよう、認証ヘルパーを設定します。

```
gcloud auth configure-docker ${REGION}-docker.pkg.dev
```

> Tips:
> ECR の `aws ecr get-login-password` に相当します。
> こちらは一度設定すれば、push のたびにトークンを取り直す必要はありません。

3. イメージにタグを付けて push する

Artifact Registry のイメージパスは `リージョン-docker.pkg.dev/プロジェクト/リポジトリ/イメージ名:タグ` という形式です。

```
docker tag handson-app:v1 ${IMAGE}:v1
docker push ${IMAGE}:v1
```

初回の push はイメージ全体を転送するため、目安として数十秒から1分ほどかかります。
push できたか確認してください。

```
gcloud artifacts docker images list ${REGION}-docker.pkg.dev/${PROJECT_ID}/${REPO} --include-tags
```

コンソールの Artifact Registry からも見えます。

成功していれば `gcloud artifacts docker images list` の出力に `.../handson/app` の行が1つ表示され、`TAGS` 列が `v1` になっています。

4. Cloud Run にデプロイする

いよいよ本番です。
このコマンド1つで、ECS なら ALB・ターゲットグループ・ACM 証明書・サービス・タスク定義を組み立てていた作業がすべて終わります(1章の発展コラムで触れた ECS Express Mode を除く)。

```
gcloud run deploy handson-app \
  --image ${IMAGE}:v1 \
  --region ${REGION} \
  --allow-unauthenticated

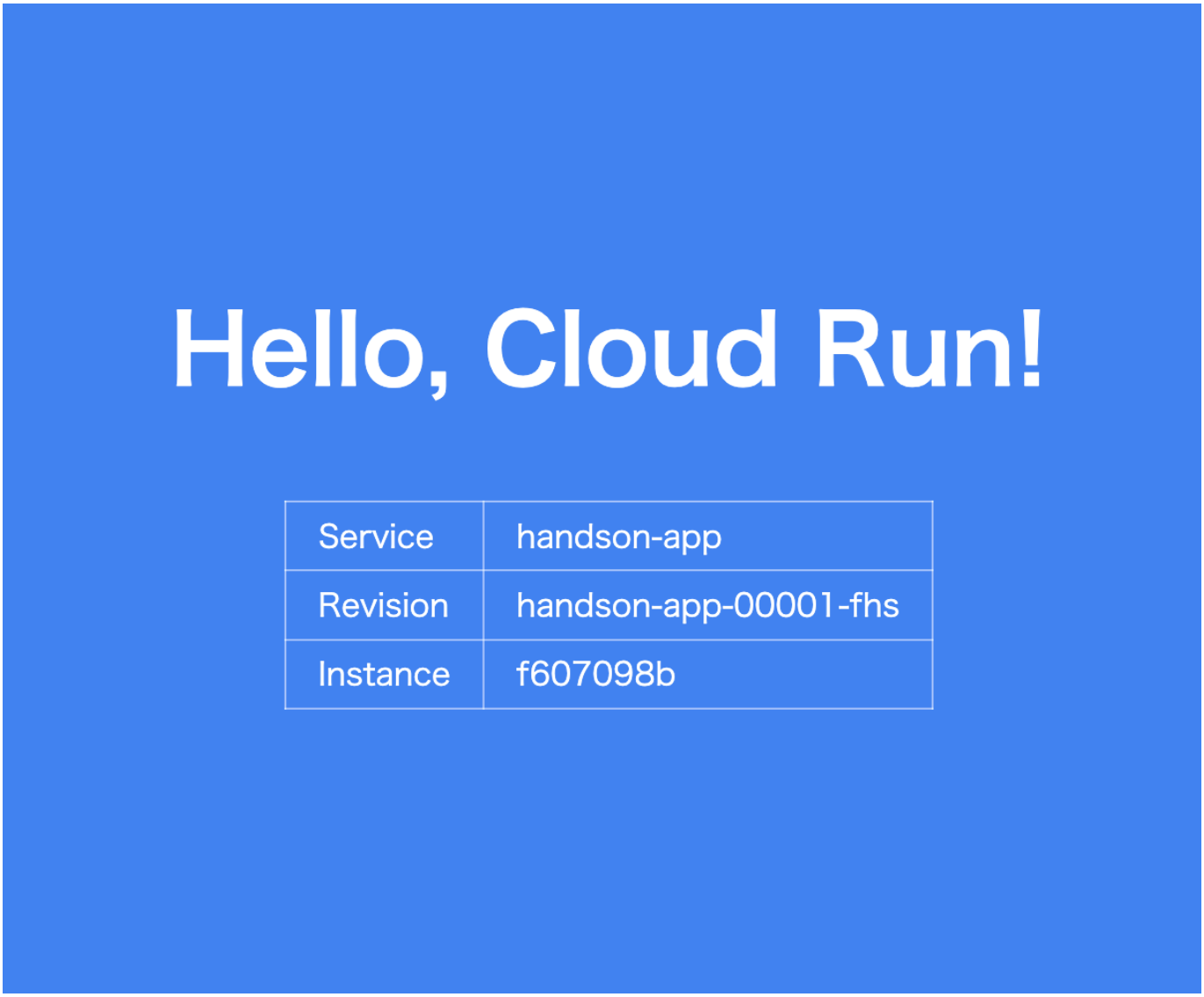
# 成功した場合、最後の行にURLが発行されます。
# Service URL: https://handson-app-123456.asia-northeast1.run.app
```

- `handson-app` が Service 名になります
- `--allow-unauthenticated` は「認証なしで公開」。
    - デフォルトは IAM 認証必須(=閉じている)で、AWS と逆のデフォルトです

30秒ほどで完了し、`Service URL: https://handson-app-<プロジェクト番号>.asia-northeast1.run.app` のような URL が表示されます。

5. アクセスして確認する

表示された URL をブラウザで開いてください。
4章で Cloud Shell の中で見たのと同じ青い画面が、今度は本物のインターネット越しに表示されます。



Cloud Shell エディタでのプレビューとの違い

- `Service` が `handson-app` に、`Revision` が `handson-app-00001-xxx` になっている
  - Cloud Run が注入する環境変数 `K_SERVICE` / `K_REVISION` をアプリが表示している
- URL が最初から **HTTPS**。証明書の発行も更新も何もしていない

コンソールの Cloud Run のページ も開いてみてください。
Service の一覧、リビジョン、メトリクス、ログがすべて1画面に集約されています。

6.「したこと」と「しなかったこと」

ここまでで「したこと」はリポジトリ作成 → push → deploy の3コマンドです。
それより「**しなかったこと**」に注目してください。

- VPC・サブネット・セキュリティグループを作らなかった
- ロードバランサーもリスナーも作らなかった
- TLS 証明書を発行しなかった
- タスク定義(CPU/メモリ/ポート)を書かなかった
- キャパシティ(台数)を決めなかった

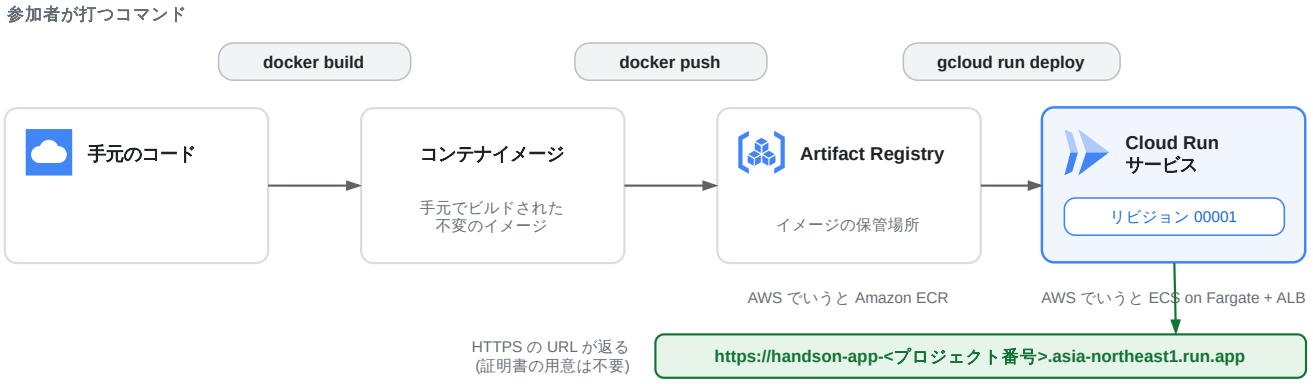
- タスク実行ロール・タスクロールを分けて作らなかった ※Service には既定のサービスアカウント(AWSで例えるとIAMロール)が1つ付くだけ

最後の1つ、サービスアカウントは確認できます。
以下のコマンドを実行すると、`<プロジェクト番号>-compute@developer.gserviceaccount.com` の形のサービスアカウントが1つだけ表示されます(コンソールでは Service の詳細画面の「セキュリティ」タブで同じものが見えます)。

```
gcloud run services describe handson-app \
  --region ${REGION} \
  --format 'value(spec.template.spec.serviceAccountName)'
```

1章で話した「ビルディングブロック vs SaaS的アプローチ」が、これで体感できたはずです。

まとめ



- Artifact Registryにコンテナイメージを保管した。
- Artifact Registryのコンテナイメージからを実行した。
- Cloud Run Serviceに実際にデプロイし、アクセスした。

6. 更新とロールバック

アプリを変更して2回目のデプロイを行い、「リビジョン」がどう積み重なるかを見ます。
その後、**本番障害を想定してロールバック** を体験します。

1. アプリを変更する (v2)

`src/index.ts` の上部にある2行の既定値(`||` の右側)を書き換えます。

```
const MESSAGE = process.env.MESSAGE || "Hello, Cloud Run v2!";
const BG_COLOR = process.env.BG_COLOR || "#EA4335"; // 赤
```

`MESSAGE` と `BG_COLOR` は環境変数から読み、設定されていなければこの既定値を使います。
後述しますが、環境変数として外から渡すことも可能です。

2. build して push して deploy

前章でやったことの繰り返しです。
タグは `v2` に変更する点に気をつけてください。

```
docker build -t ${IMAGE}:v2 .
docker push ${IMAGE}:v2

gcloud run deploy handson-app \
  --image ${IMAGE}:v2 \
  --region ${REGION}
```

> 2回目以降は `--allow-unauthenticated` などの設定を繰り返す必要はありません。
> Service の設定は維持され、**変更したもの(イメージ)だけが新しいリビジョンとして記録されます。**

ブラウザをリロードしてください。
画面が赤くなり、Revision が `handson-app-00002-xxx` に変わっていれば成功です。
また、ダウンタイムなしで切り替わったことにも注目してください。

環境変数

コマンドから環境変数を設定する

`gcloud run deploy` に `--update-env-vars` を付けると、デプロイと同時に環境変数を設定できます。

```
gcloud run deploy handson-app \
  --image ${IMAGE}:v2 \
```

```
  --region ${REGION} \
  --update-env-vars '^@^MESSAGE=Hello from env!@BG_COLOR=#FBBC04'
```

- `--update-env-vars` は指定した変数だけを追加・上書きし、ほかの環境変数はそのまま残します
- 似たフラグの `--set-env-vars` は、既存の環境変数をすべて消してから設定します
- 先頭の `^@^` は、変数どうしの区切り文字をカンマから `@` に変える指定です
  - gcloud は値をカンマで区切って読むため、`Hello, Cloud Run!` のようにカンマを含む値はそのままだと途中で分かれ、`Bad syntax for dict arg` のエラーになります
  - 区切り文字には、どの値にも出てこない文字を選びます(値に `@` が入るなら `^:^` などにする)
- 引数全体をシングルクォートで囲むのは、`#`・`!`・空白をシェルに解釈させず、そのまま gcloud に渡すためです

カンマを含む値を設定する例です。

```
gcloud run deploy handson-app \
  --image ${IMAGE}:v2 \
  --region ${REGION} \
  --update-env-vars '^@^MESSAGE=Hello, Cloud Run!@BG_COLOR=#FBBC04'
```

デプロイが終わったらブラウザをリロードしてください。
画面が黄色になり、Revision も新しい名前に変わります。
環境変数の変更も設定の変更なので、新しいリビジョンが作られます (リビジョン = イメージ + 設定のスナップショット)。

コンソールからから環境変数を設定する

コンソールからも環境変数の設定が可能です。

1. Cloud Run の「サービス」から `handson-app` を選び、「新しいリビジョンの編集とデプロイ」をクリック
2. 「コンテナ」タブの「変数とシークレット」で、環境変数を追加
   - 名前 `MESSAGE` / 値 `Hello from env!`
   - 名前 `BG_COLOR` / 値 `#FBBC04` (黄)
3. 「デプロイ」をクリック

3. リビジョンの一覧を見る

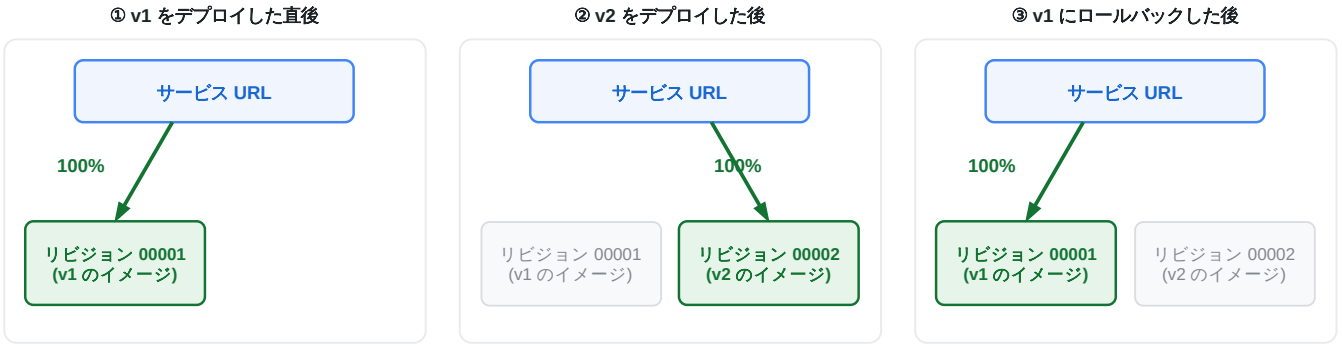
```
gcloud run revisions list --service handson-app --region ${REGION}
```

`00001` (青/v1)と `00002` (赤/v2)の両方が残っています。
リビジョンは **イメージ+環境変数+リソース設定を固めた不変のスナップショット** で、デプロイのたびに積み重なっていきます。

> **AWSとの比較**
> ECS のタスク定義リビジョンに似ていますが、Cloud Run のリビジョンは「どこにトラフィックを流すか」の制御と一体化し

> ている点が違います。タスク定義+ALB加重ルーティング+デプロイ履歴が1つの概念になっているイメージです。

4. ロールバックする



リビジョンの箱は消えず、増えるだけ。ロールバックで動くのは矢印(トラフィックの割当)だけで、再デプロイは起きない。
番号は概念として描いています。実際の採番は連番にならず飛ぶことがあります。

ここで想定シナリオ:**「v2 をリリースしたらバグ報告が来た!すぐ戻したい!」**

AWS ならどうしますか?
前のタスク定義を指定してサービス更新、デプロイが走るのを待ち、数分かかります。
Cloud Run では、**すでに存在する v1 のリビジョンにトラフィックを振り向けるだけ**です。
新しいデプロイは走りません。

v1 のリビジョン名を確認

```
gcloud run revisions list --service handson-app --region ${REGION}


# REVISION: handson-app-00002-c58
#  :
# REVISION: handson-app-00001-fhs
```

トラフィックをv1に100%戻します。
`handson-app-00001-<xxx>` のsuffix部分はランダムなので、自分のlistコマンドのREVISION名に書き換えてください。

```
export REV=handson-app-00001-<xxx>
gcloud run services update-traffic handson-app \
  --region ${REGION} \
  --to-revisions ${REV}=100
```

数秒待ってからブラウザをリロードしてください。**青(v1)に戻ります。**
トラフィックの切り替えは瞬時ではないため、まだ赤のままなら少し待って再読み込みしてください。

コンソールでも同じことができます: Cloud Run → サービス → 「リビジョン」タブ → 対象リビジョンの右側にある省略記号アイコンから「トラフィックを管理」。
障害対応の現場では GUI でポチッと戻せるのは心強いです。

v2 に戻す

ロールバック体験が終わったので、最新リビジョン(v2)に戻しておきます。

```
gcloud run services update-traffic handson-app \
  --region ${REGION} \
  --to-latest
```

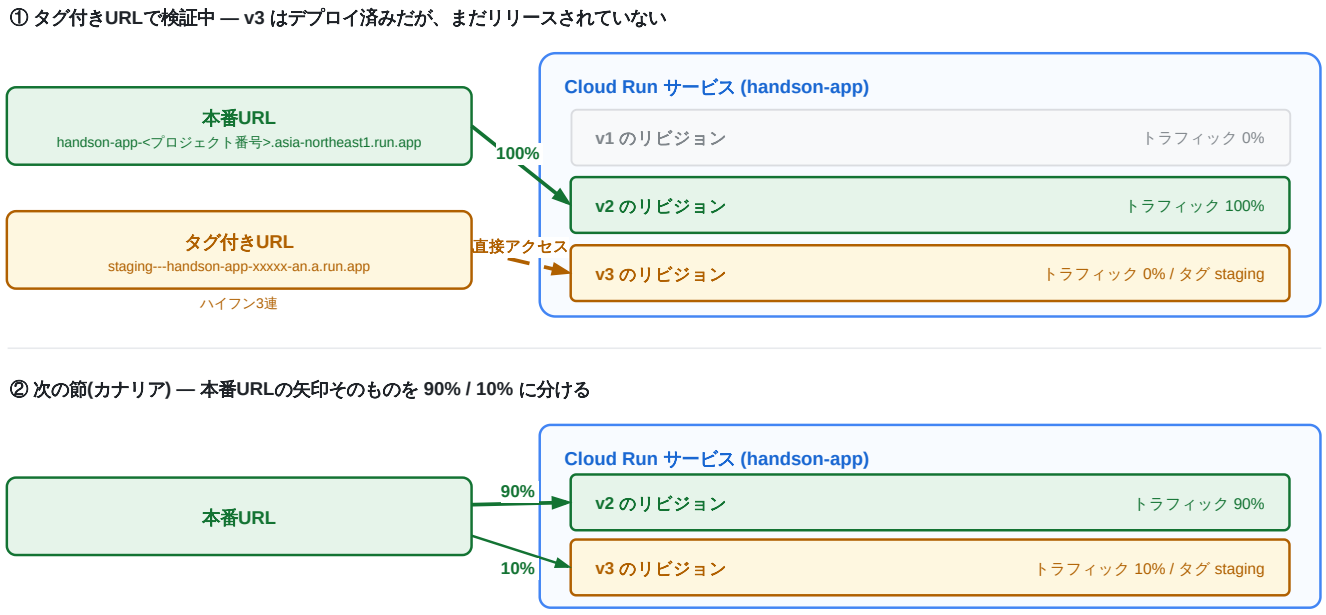
`--to-latest` で最新のリビジョンに切り替えることができます。
ブラウザで赤(v2)に戻ったことを確認してください。

まとめ

- デプロイのたびに不変のリビジョンが積み重なる
- ロールバック ＝ 過去リビジョンへのトラフィック切り替え。**再デプロイ不要、数秒で完了**
- 「イメージの不変性」がプラットフォーム機能として活きている

7. カナリアリリースとタグ付きURL

前章のトラフィック制御を一歩進めて、**新バージョンを10%のユーザーにだけ出すカナリアリリース**と、**トラフィックを流さずに本番環境で動作確認できるタグ付きURL**を体験します。



> **AWSとの比較**
> これを ECS でやるには CodeDeploy のブルー/グリーン+ALB の加重ターゲットグループの設定が必要でした(検証用リスナーは任意ですが、リリース前検証をしたいなら実質必要になります)。Cloud Run では **Service に元から組み込まれた機能**です。

1. 3つ目のリビジョンを作る

今回はイメージを作り直さず、6章のTipsで紹介したCLI(/コンソール)から環境変数でメッセージと背景を変えたものを新しいリビジョンとします。

イメージは v2 のまま、`MESSAGE` を `Hello, Cloud Run v3`、`BG_COLOR` を緑(`#34A853`)にします。

`--no-traffic` を付けてデプロイすると、リビジョンは作られますが**ユーザーへのトラフィックは1%も流れません。**
さらに `--tag` を付けると、そのリビジョン専用のURLが発行されます。

```
gcloud run deploy handson-app \
  --image ${IMAGE}:v2 \
  --region ${REGION} \
  --no-traffic \
  --tag staging \
  --update-env-vars '^@^MESSAGE=Hello, Cloud Run v3@BG_COLOR=#34A853'
```

出力の最後に、このリビジョン専用のタグ付きURLが表示されます(`--no-traffic` のときは `Service URL:` の行は出ません)。
本番URLは5章で取得したものと同じです。

- 本番URL: `https://handson-app-<プロジェクト番号>.asia-northeast1.run.app` → **まだ赤(v2)のまま**
- タグ付きURL: `https://staging---handson-app-xxxxx-an.a.run.app` → **緑(v3)**

両方をブラウザで開いて確認してください。
本番と同じ環境・同じ設定で、リリース前のバージョンだけを検証できるURLが手に入りました。
ステージング環境を別に組む代わりに、本番 Service の中に検証チャネルを持てるということです。

成功していれば

出力に `serving 0 percent of traffic` と `The revision can be reached directly at https://staging---...` の2点が出ます。
タグ付きURLは緑(v3)、本番URLは赤(v2)のままです。URLを見失ったら `gcloud run services describe handson-app --region ${REGION}` の出力で本番URLとタグ付きURLの両方を確認できます。
なお**リビジョン名の番号は 00003 にならないことがあります**。番号の採番はデプロイ回数と一致せず、`handson-app-00005-xxx` のように飛ぶことがあります(リビジョンを見分けるのは末尾のランダム文字列です)。番号が飛んでいても失敗ではありません。

2. 10%だけ流す(カナリアリリース)

v3 の動作確認が取れた想定で、まず10%のユーザーにだけ出します。

```
gcloud run services update-traffic handson-app \
  --region ${REGION} \
  --to-tags staging=10
```

確認してください。
`/api` エンドポイントを20回叩いて、どのリビジョンが応答したかを集計します。

```
URL=$(gcloud run services describe handson-app --region ${REGION} --format 'value(status.url)')

for i in $(seq 1 20); do curl -s ${URL}/api | jq -r .revision; done | sort | uniq -c
```

リビジョン名が2種類表示され、`18 handson-app-00002-vjk` のように v2 のリビジョンが18回・v3 のリビジョンが2回、という比率になるはずです。
ブラウザを何度もリロードして「たまに緑が出る」のを見るのも楽しいです。

> 実務ではこの状態でエラーレートやレイテンシのメトリクス(9章)を監視し、問題なければ段階的に割合を増やしていきます。

成功していれば

`gcloud run services describe handson-app --region ${REGION}` の Traffic 欄が `90% handson-app-00002-xxx` と `10% handson-app-000xx-xxx` (その下に `staging: https://staging---...` が付きます)に分かれ、20回の集計でも v3 のリビジョン名が数回混ざります。
10%の乱数なので v3 のリビジョン名が0回や4回になることもあり、比率が多少ずれても失敗ではありません。

3. 100%に昇格する

```
gcloud run services update-traffic handson-app \
  --region ${REGION} \
  --to-latest
```

> `--to-latest` は「いま動いているリビジョンに固定する」という意味ではなく、**`LATEST` (＝最新リビジョン)に100%割り当てる**という設定です。
> そのため、この状態で次のデプロイを行うと、新しく作られたリビジョンがそのまま `LATEST` になってトラフィックを受け取ります(5〜6章で体験した普通のデプロイ挙動に戻る、ということです)。

ブラウザで全リロードが緑(v3)になれば完了です。
もし v3 に問題が見つかっていたら?——前章でやった通り、v2 に一瞬で戻せます。

成功していれば

Traffic 欄に `100% LATEST (currently handson-app-000xx-xxx)` が現れ、上の for ループを再実行すると20回すべて v3 のリビジョン名になります。
`staging` タグ自体は残るので、Traffic 欄には `0% (currently -) handson-app-000xx-xxx` と `staging (Adding):` / `staging (Deleting): https://staging---...` という行も並びます。
これは表示上の見え方で、トラフィックは 100% 最新リビジョンに向いています(for ループの結果が20回すべて同じ v3 のリビジョン名ならそれが答えです)。

まとめ

- `--no-traffic` + `--tag` で「デプロイ」と「リリース」を分離できる
- タグ付きURLで、本番環境そのものを使ったリリース前検証ができる
- カナリア → 段階的昇格 → 即ロールバック、が追加インフラなしで完結する

デプロイのたびに緊張するチームほど、この機能の価値は大きいはずです。

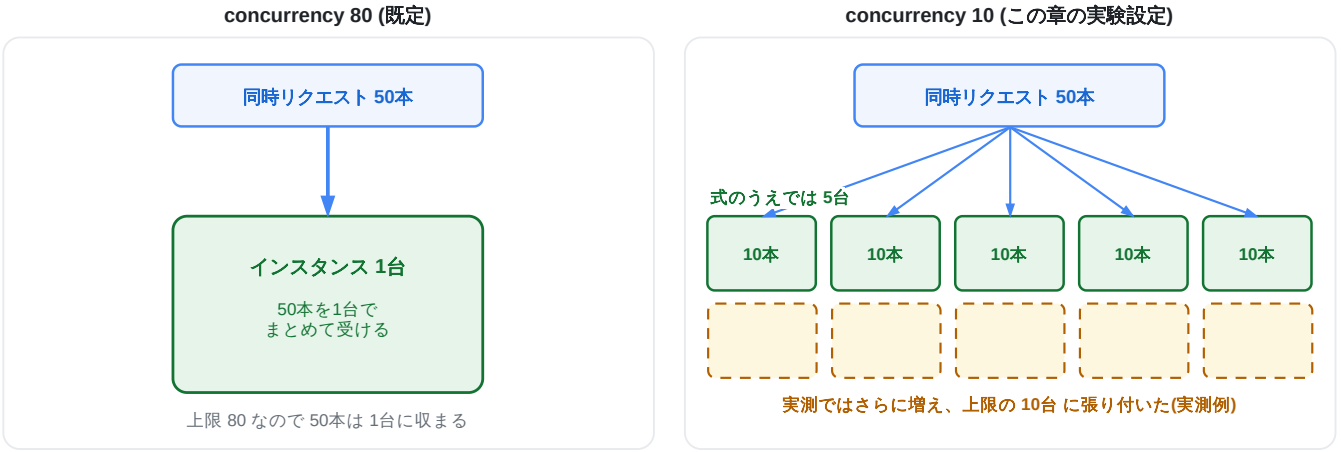
8. オートスケールを観察する

6〜7章ではリビジョンを積み重ね、どのバージョンに何%流すかを切り替えました。

この章で見るのはインスタンスの数、リクエストが増えたときに何台で受けるかです。
Cloud Run の非機能まわりの主役、オートスケールを実際に負荷をかけて観察します。

あわせてコールドスタートとその対策も体験します。

Cloud Run のスケールの仕組み



オートスケーラーは CPU 使用率と同時実行数の目標を 60% に置いて増減させるため、台数は当てるものではなく観察するもの。

参考: 標準の Lambda は 1リクエスト = 1実行環境なので、同じ50本には 50個の実行環境が要る。

Cloud Run は**同時に処理しているリクエスト数(concurrency)とCPU使用率**を基準にインスタンス数を自動調整します。

```
必要インスタンス数 ≒ 同時リクエスト数 ÷ concurrency(1台が同時に受ける数)
```

- この式は**キャパシティの直感をつかむための概算**です。実際のオートスケーラーは同時リクエスト数とCPU使用率の平均を見て、**どちらも既定で目標値60%以下に収まるように**台数を決めるため、計算どおりの整数台にはなりません(たいてい式より多めの台数になります)
- `concurrency` は1 vCPU構成なら既定80(gcloud / Terraform で新規作成した場合は「vCPU数 × 80」)。1インスタンスあたり最大1000まで上げられます。**標準のLambda(1リクエスト=1環境)と違い、1台で複数リクエストを捌く**
- トラフィックが来なくなると0台までスケールインする(スケールtoゼロ)。ただし縮小は即座ではなく、**次のスパイクに備えて最大15分ほどインスタンスがアイドルのまま残ります**
- ECS のように Application Auto Scaling でスケーリングポリシーを登録する必要はなく、**設定不要で最初から動いている**(ECS もターゲット追跡ポリシーを使えば CloudWatch アラーム自体は AWS 側が自動生成してくれますが、ポリシーを登録する作業そのものは必要です)

1. 観察しやすいように設定を変える

デフォルトの concurrency 80 だとなかなかスケールしないので、実験用に1台あたり10接続に絞り、上限を10台にします。

```
gcloud run services update handson-app \
  --region ${REGION} \
  --concurrency 10 \
  --max-instances 10
```

成功していれば

`gcloud run services update` の出力に新しいリビジョン名(`handson-app-000xx-xxx`)が出て完了し、`gcloud run services describe handson-app --region ${REGION} --format 'value(spec.template.spec.containerConcurrency)'` が `10` を返します。

> この操作でも新しいリビジョンが作られます。リビジョン＝イメージ＋**設定**のスナップショットだからです。

2. 負荷をかける

負荷試験スクリプトを作ります。
Node.js の標準APIだけで動く31行のスクリプトで、外部ツールのダウンロードは不要です(Cloud Shell には Node.js が入っています)。

```
cat > ~/load.mjs <<'EOF'
// 簡易負荷試験スクリプト。Node.js 標準APIのみで動く(外部バイナリのダウンロード不要)。
//   node load.mjs <URL> [同時接続数=50] [継続秒数=30]
const [url, concurrency = "50", duration = "30"] = process.argv.slice(2);

if (!url) {
  console.error("usage: node load.mjs <URL> [concurrency] [durationSec]");
  process.exit(1);
}

const until = Date.now() + Number(duration) * 1000;
let ok = 0;
let ng = 0;

const worker = async () => {
  while (Date.now() < until) {
    try {
      // タイムアウトを入れないと、サーバー無応答時にワーカーが張り付いて
      // 指定した継続秒数を過ぎても終わらなくなる
```

```
      const res = await fetch(url, { signal: AbortSignal.timeout(10000) });
      await res.arrayBuffer(); // レスポンスを読み切って接続を再利用する
      if (res.ok) ok++;
      else ng++;
    } catch {
      ng++;
    }
  }
};

console.log(`load: ${url} concurrency=${concurrency} duration=${duration}s`);
await Promise.all(Array.from({ length: Number(concurrency) }, worker));
console.log(`done: success=${ok} failure=${ng}`);
EOF
```

アプリの `/heavy` は1秒かかる擬似的に重いエンドポイントです。
同時50接続で30秒間叩きます。

実行する前に、何台までスケールアウトするか予想してみてください。
concurrency を10に絞ったので、1台で受けられる同時リクエストは10。
50接続なら式のうえでは5台です。

ただしオートスケーラーは同時リクエスト数を目標値(既定60%)以下に保とうとするため、実際には5台より多く、8台前後や上限の10台まで立ち上がることもあります。
台数は当てるものではなく観察するものです。
予想と違ったら「なぜ?」を考えるのがこの実験の面白いところです。

```
URL=$(gcloud run services describe handson-app --region ${REGION} --format 'value(status.url)')

node ~/load.mjs ${URL}/heavy 50 30
```

3. 観察する

負荷をかけている間(および直後)に、2つの方法で観察します。

コンソールで: Cloud Run → `handson-app` →「指標」タブ。
「コンテナ インスタンスの数」がゼロから跳ね上がり、負荷を止めたあと最大15分ほどかけてまたゼロに戻っていく様子が見えます。
リクエスト数・レイテンシ(p50/p95/p99)・CPU使用率も**何も設定していないのに**最初から揃っています。

手元で: `/api` はインスタンスごとに違う `instance` ID を返します。
何台に分散しているか数えてください。

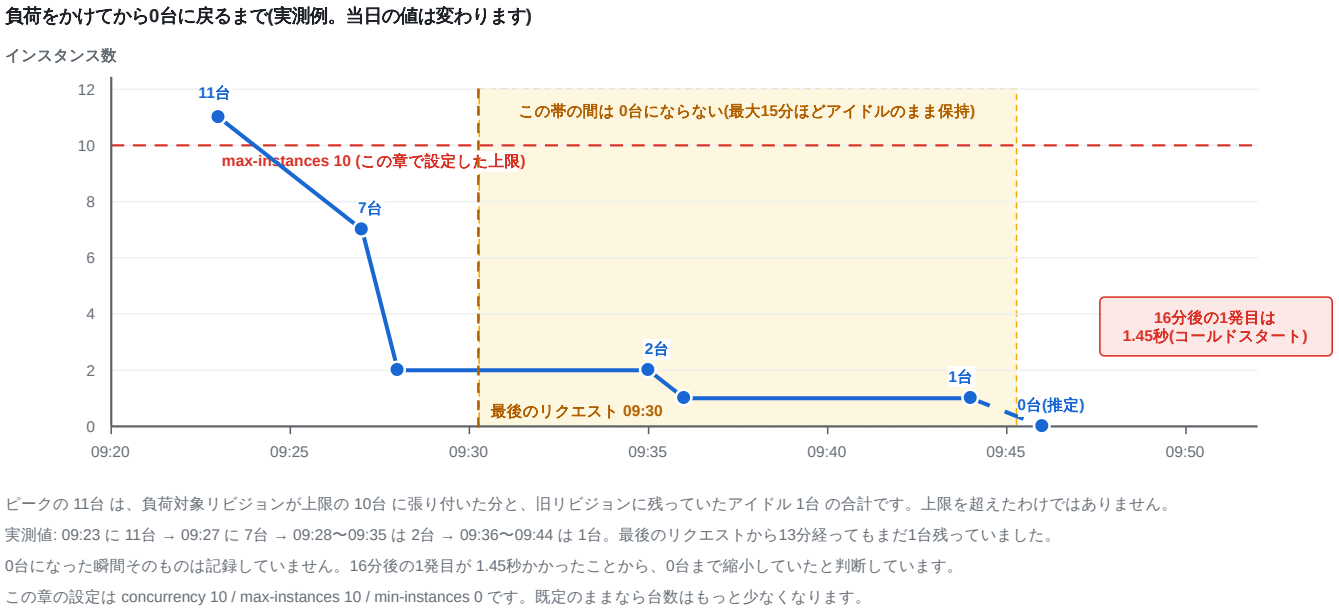
```
for i in $(seq 1 30); do curl -s ${URL}/api | jq -r .instance; done | sort | uniq -c
```

複数の instance ID が出てくれば、リクエストが複数台に分散している証拠です。
負荷を止めてから数分〜15分ほど待って同じコマンドを打つと、1台(やがて0台)に戻ります。

成功していれば

`node ~/load.mjs` の最後に `done: success=<数百〜数千> failure=0` (起動中のリクエストが少数失敗することはあります)が表示され、`sort | uniq -c` の出力に **2つ以上の instance ID** が並びます。コンソールの「コンテナ インスタンスの数」もゼロ(または1台)から複数台へ跳ね上がっています。台数は実行ごとに変わるので、5台でなくても失敗ではありません。

4. コールドスタートと min-instances



ピークの 11台 は、負荷対象リビジョンが上限の 10台 に張り付いた分と、旧リビジョンに残っていたアイドル 1台 の合計です。上限を超えたわけではありません。
実測値: 09:23 に 11台 → 09:27 に 7台 → 09:28〜09:35 は 2台 → 09:36〜09:44 は 1台。最後のリクエストから13分経ってもまだ1台残っていました。
0台になった瞬間そのものは記録していません。16分後の1発目が 1.45秒かかったことから、0台まで縮小していたと判断しています。
この章の設定は concurrency 10 / max-instances 10 / min-instances 0 です。既定のままなら台数はもっと少なくなります。

スケールtoゼロの代償が**コールドスタート**です。
0台の状態への最初のリクエストは、コンテナ起動を待つ分だけ遅くなります。

Cloud Run は最大15分ほどインスタンスをアイドルのまま残すので、0台に戻るのを待つには**15分以上**放置する必要があります。
待っている間は9章・10章を先に進めて、最後にここへ戻って下の `curl` を打つのがおすすめです。
それでも1回目と2回目に差が出ないことがありますが、直前の負荷でインスタンスが温まったまま残っているだけなので失敗ではありません。

```
curl -s -o /dev/null -w "time_total: %{time_total}s\n" ${URL}/
curl -s -o /dev/null -w "time_total: %{time_total}s\n" ${URL}/
```

1回目だけ遅い(このアプリは軽いですが、0台の状態からだと1秒〜1.5秒ほどかかります。重いアプリではさらに長くなります)のがわかります。
対策は常時1台温めておくことです。

```
gcloud run services update handson-app \
  --region ${REGION} \
  --min-instances 1
```

> **トレードオフ:** min-instances を設定した分はアイドル時も課金されます(リクエスト課金の場合、アイドル中は低い単価が適用されます)。「完全従量制(コールドスタートあり)」と「最低台数の固定費(コールドスタートを起きにくくする)」を、**Service ごとにダイヤルで選べる**のが Cloud Run の良いところです。Lambda の Provisioned Concurrency と ECS の常駐、両方の選択肢が1つの Service に入っていると考えてください。なお min-instances は「温かいインスタンスを保つベストエフォート」であり、コールドスタートがゼロになる保証ではありません。

確認できたら、課金を避けるため戻しておきます。

```
gcloud run services update handson-app \
  --region ${REGION} \
  --min-instances 0
```

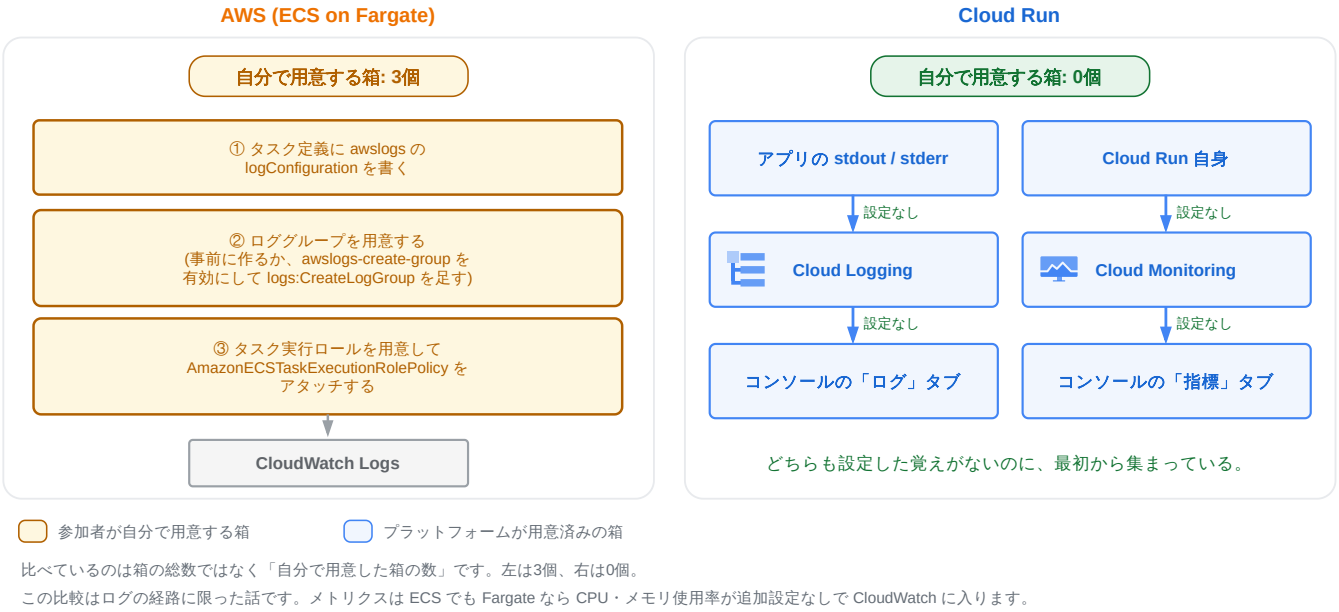
成功していれば

1回目の `curl` の `time_total` が2回目より明確に大きくなります(このアプリなら1回目が1秒〜1.5秒、2回目は数十ms程度)。
`--min-instances` の更新はどちらも新しいリビジョン名を出力して完了し、`gcloud run revisions list --service handson-app --region ${REGION}` の行が増えます。

まとめ

- スケールの基準は同時リクエスト数とCPU使用率。台数は「同時リクエスト数 ÷ concurrency」で概算でき、設定ゼロで動く
- スケールtoゼロ ⇔ コールドスタートはトレードオフ。`min-instances` で調整(縮小には最大15分のアイドル時間がある)
- `max-instances` は暴走課金・下流(DB)保護の安全弁としても重要

9. ログとメトリクスをのぞく



比べているのは箱の総数ではなく「自分で用意した箱の数」です。左は3個、右は0個。
この比較はログの経路に限った話です。メトリクスは ECS でも Fargate なら CPU・メモリ使用率が追加設定なしで CloudWatch に入ります。

短い章です。
ここまでのハンズオンで一度も「ログの設定」をしていないのに、実はすべて記録されています。
それを見に行きます。

1. ログを見る

Cloud Run のコンソール → `handson-app` →「ログ」タブを開いてください。

- コンテナが stdout/stderr に出しただけのログが、すべて収集されている
- アプリが出していた `{"severity": "INFO", "message": "index accessed", ...}` という**1行JSONが構造化ログとして解釈され**、severity で色分けされている

CLI からも見られます:

```
gcloud run services logs read handson-app --region ${REGION} --limit 20
```

成功していれば

`2026-01-01 12:34:56 GET 200 https://handson-app-.../` のようなリクエストログと、`listening on port 8080` のようなアプリの出力が時刻順に並びます。

ただし `gcloud run services logs read` はログ本文を**プレーンテキストとしてしか表示しない**ため、アプリが出した1行JSON(`index accessed`)は**時刻だけの空行**に見えます。JSON の中身まで CLI で見たいときはこちらを使ってください。

```
gcloud logging read 'resource.type="cloud_run_revision" AND resource.labels.service_name="handson-app"
AND jsonPayload.message="index accessed"' --limit 5 --format 'yaml(timestamp,severity,jsonPayload)'
```

> **AWSとの比較:** CloudWatch Logs へ送るには awslogs ログドライバやロググループの設定、IAM 権限が必要でした。Cloud Run では**コンテナが stdout に書く、以上**です。構造化ログにしたければ1行JSONにするだけで、フィールド検索・severityフィルタが効くようになります。

2. Logs Explorer で検索する

「ログ」タブの上部から「ログ エクスプローラで表示」を開くと、プロジェクト全体のログを横断検索できます。
クエリ欄に以下を入れてみてください:

```
resource.type="cloud_run_revision"
resource.labels.service_name="handson-app"
jsonPayload.message="index accessed"
```

アプリが出した構造化ログだけに絞り込めます。
`jsonPayload.instance` など、`log()` 関数で自分が入れたフィールドもそのまま検索条件に使えます。

成功していれば

`index accessed` の行だけが残ります。

3. メトリクスを見る

「指標」タブ(8章でも見ました)には以下が最初から揃っています:

- リクエスト数 / レイテンシ(p50, p95, p99)
- コンテナインスタンス数 / 課金対象インスタンス時間
- CPU / メモリ使用率

実務では、7章のカナリアリリース中にこの画面を開き、「新リビジョンだけエラー率が上がっていないか」を見ながら昇格判断をします。
リビジョン別にメトリクスをフィルタできることも確認してみてください。

成功していれば

リクエスト数のグラフに、ここまでのアクセス分が現れます。

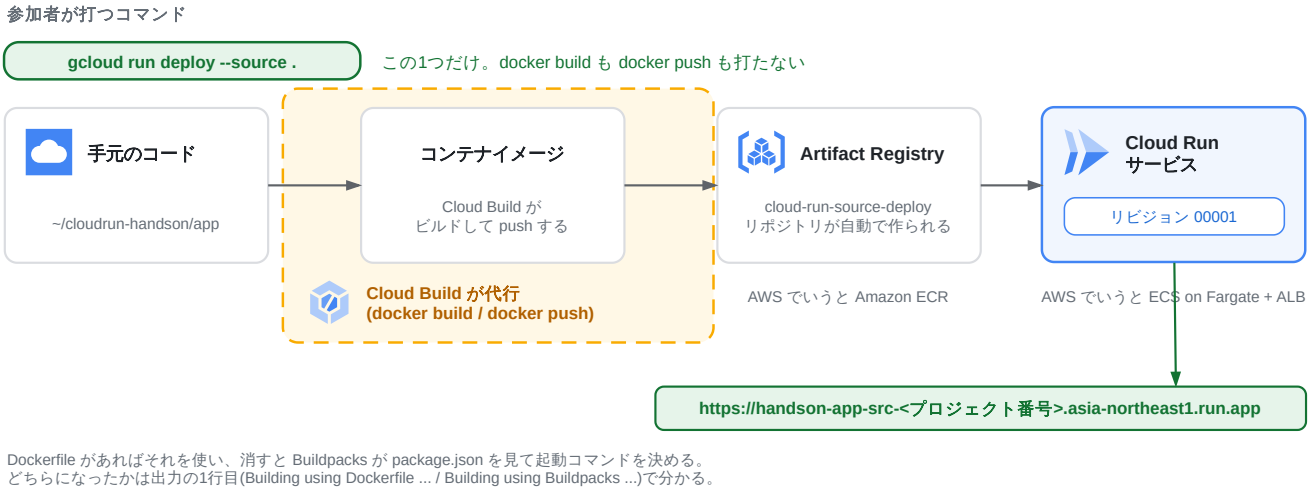
まとめ

- ログもメトリクスも「設定する」のではなく「最初からある」
- アプリ側の作法は「stdout に(できれば1行JSONで)書く」だけ
- 1章で話した SaaS 的アプローチは、オブザーバビリティにも一貫している

10. ソースデプロイ

最後に、今日学んだ手順を巻き戻すような機能を紹介します。
ソースコードだけで Cloud Run にデプロイする方法です。

1. ソースデプロイ



`--image` の代わりに `--source` を指定すると、Cloud Build がソースからイメージをビルドして GAR に push し、そのままデプロイまで行います。

```
cd ~/cloudrun-handson/app
gcloud run deploy handson-app-src \
  --source . \
  --region ${REGION} \
  --allow-unauthenticated
```

数分待つと、別の Service `handson-app-src` として同じアプリが公開されます。

成功していれば

ビルドとデプロイに数分かかったあと、`Service URL: https://handson-app-src-...` が表示されます。
開くと5章と同じレイアウトの画面が出ますが、**色とメッセージは赤の v2** です(6章で書き換えた `src/index.ts` からビルドされます。
7章の v3 は `handson-app` に設定した環境変数で作ったもので、別の Service である `handson-app-src` には引き継がれません)。
`Service` の欄が `handson-app-src` になっていることも確認してください。初回はビルド済みイメージの置き場として `cloud-run-source-deploy` という Artifact Registry リポジトリが自動で作られます(確認を求められたらそのまま進めてください)。途中で出力が止まって見えても、ビルド待ちなので待って大丈夫です。

このディレクトリには Dockerfile があるのでそれが使われますが、**Dockerfile を消しても動きます**。
その場合は Google Cloud の Buildpacks が `package.json` を見て「Node.js アプリだな」と判断し、

`scripts.start`(この教材では `node src/index.ts`)を起動コマンドにしたコンテナイメージを自動生成します。

Dockerfile を消して同じ `gcloud run deploy handson-app-src --source . --region ${REGION} --allow-unauthenticated` を実行しても、やはり数分後に同じ画面が返ってきます。
切り替わったかどうかは出力の1行目で分かります。Dockerfile があるときは `Building using Dockerfile and deploying container to Cloud Run service [handson-app-src]`、消したあとは `Building using Buildpacks and deploying container to Cloud Run service [handson-app-src]` になります。

Dockerfile を消したくない場合は、この節は読むだけで先に進んでも構いません。

2. では、なぜ今日 Dockerfile から始めたのか

「最初からコンテナイメージをbuild/pushせずにこれを教えてくれれば良かったのに」と思うかもしれません。
このハンズオンの順番には意図があります。

1. ソースデプロイは**中でやっていることが今日の手順そのもの**です
    - build → push → deploy。
    - 仕組みを知った上で使う自動化と、知らずに使う魔法は違います
2. 実務では Dockerfile を書く場面が必ず来ます
    - 依存OSパッケージ、マルチステージビルド、社内ベースイメージ…
3. 逆に、プロトタイプや社内ツールなら「Dockerfile なしでとりあえず公開」で十分なこともあります。

手軽さの階段を自分で選べるのが Cloud Run です:

```
gcloud run deploy --source .          # ソースだけ(Buildpacks におまかせ)
gcloud run deploy --source .          # ソース+ Dockerfile(ビルドはおまかせ)
docker build && push && deploy        # 全部自分で制御(今日やったこと)
+ Cloud Build トリガー                # git push で自動デプロイ(CI/CD)
```

まとめ

- Cloud Runはコンテナイメージだけでなく、ソースコードのみでもデプロイ可能。
- ソースコードデプロイはGoogle Cloudがかわりにコンテナ化をしてくれているラッパーのようなもの。
- Cloud Runの実務ではDockerfile(そして今回取り扱わなかったCloud Runのyaml(IaC))を利用する。

99. 後片付け

お疲れさまでした!
リソースの片付けを行います。

Cloud Shell を開き直していると `${PROJECT_ID}` や `${REGION}` などの環境変数が消えているので、先に 5章の「環境変数の準備」を実行し直してください(1. と 2. のどちらを選ぶ場合も必要です)。

1. おすすめ: プロジェクトごと削除

ハンズオン専用プロジェクトで進めた場合は、これが一番確実です。

まず削除対象のプロジェクトIDを表示して確認します。

```
echo ${PROJECT_ID}
```

表示されたIDがハンズオン専用プロジェクトであることを確認してから実行してください(空の場合は冒頭の環境変数の準備をやり直してください)。

```
gcloud projects delete ${PROJECT_ID}
```

(30日間は復元可能な状態で保留されたあと、完全に削除されます)

2. 個別に削除する場合

既存プロジェクトを使った場合はこちら。

```
# Cloud Run Service
gcloud run services delete handson-app --region ${REGION} --quiet
gcloud run services delete handson-app-src --region ${REGION} --quiet   # 10章を実施した場合

# Artifact Registry(イメージの保管料がかかるため忘れずに)
gcloud artifacts repositories delete ${REPO} --location ${REGION} --quiet
# 10章のソースデプロイは cloud-run-source-deploy というリポジトリを自動で作るため、こちらも削除します
gcloud artifacts repositories delete cloud-run-source-deploy --location ${REGION} --quiet   # 10章を実施した場合

# 10章のソースデプロイは Cloud Build 用のバケットも自動で作ります(アップロードしたソースが残ります)
#   このバケットは `--region` に追従せず US マルチリージョンに作られるので、`gcloud storage ls` で探すときはリージョンで絞らないでください。
gcloud storage rm --recursive gs://${PROJECT_ID}_cloudbuild --quiet   # 10章を実施した場合
```

10章を実施していなければ、行末に「10章」と注記した3行は飛ばしてください(実行しても NOT_FOUND エラーになるだけで実害はありません)。

課金ポイントの整理

Cloud Run はスケールtoゼロなので、`min-instances` を0に戻してあれば放置してもほぼ課金されません(1以上のままだとアイドル状態でも課金され続けます)。
継続課金になり得るのは **Artifact Registry のストレージ**(無料枠は課金アカウントあたり0.5GB/月)と **min-instances** です。

消し忘れが心配な人へ

課金レポートで数日後に確認するか、予算アラート(Budgets & alerts)を設定しておくと安心です。AWS の Budgets と同じ感覚で使えます。